

**FACULDADE DE TECNOLOGIA DA ZONA LESTE
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS**

**LUCAS SIQUEIRA SOARES
PEDRO MENDES DA SILVA
VITOR LOPES MARTIN ESTEVES
WALLACE MARTINS VIEIRA**

**SISTEMA DE ALUGUEL DE TRAJE A RIGOR
COM FOCO EM TESTES DE SOFTWARE**

**São Paulo
2026**

LUCAS SIQUEIRA SOARES
PEDRO MENDES DA SILVA
VITOR LOPES MARTIN ESTEVES
WALLACE MARTINS VIEIRA

SISTEMA DE ALUGUEL DE TRAJE A RIGOR
COM FOCO EM TESTES DE SOFTWARE

Trabalho de Conclusão de Curso
apresentado como exigência para
obtenção do título de Tecnólogo em
Análise e Desenvolvimento de Sistemas,
na Faculdade de Tecnologia da Zona
Leste - FATEC-ZL.

Orientador: Prof. MSc. Edson Saraiva de
Almeida

São Paulo

2026

LUCAS SIQUEIRA SOARES
PEDRO MENDES DA SILVA
VITOR LOPES MARTIN ESTEVES
WALLACE MARTINS VIEIRA

SISTEMA DE ALUGUEL DE TRAJE A RIGOR
COM FOCO EM TESTES DE SOFTWARE

Trabalho apresentado ao curso de Análise e Desenvolvimento de Sistemas da Faculdade de Tecnologia da Zona Leste - FATEC-ZL, no período vespertino, para a disciplina de Trabalho de Graduação II.
Orientador: Prof. MSc. Edson Saraiva de Almeida

Aprovado em: ____ / ____ / 2026

Banca Examinadora

Prof. MSc. Edson Saraiva de Almeida - Orientador - FATEC-ZL

Julgamento: _____ Assinatura: _____

Prof. _____ FATEC-ZL

Julgamento: _____ Assinatura: _____

Prof. _____ FATEC-ZL

Julgamento: _____ Assinatura: _____

*Dedicamos este trabalho aos nossos familiares, que
nos apoiaram durante toda a trajetória acadêmica;
aos nossos professores, que compartilharam
conhecimento e nos guiaram nesta jornada; e à
comunidade de desenvolvedores de software, que
constantemente nos inspira a buscar a excelência
técnica.*

AGRADECIMENTOS

Chegamos ao momento de expressar nossa gratidão a todos que, de alguma forma, tornaram possível a conclusão desta jornada acadêmica e a realização deste Trabalho de Conclusão de Curso. Foram anos intensos de estudo, dedicação e, principalmente, descobertas que transformaram a maneira como enxergamos o desenvolvimento de software e a engenharia de testes.

Nossa eterna gratidão às nossas famílias. Aos nossos pais, mães, irmãos e demais familiares, que foram a base sólida que nos permitiu seguir em frente mesmo nos momentos mais difíceis. Sua paciência, seu incentivo e seu apoio incondicional são a razão pela qual hoje podemos celebrar esta conquista. Sabemos que cada noite de estudo até tarde, cada fim de semana dedicado ao TCC e cada período de ausência foram compreendidos por vocês com amor e empatia. Este trabalho é, em grande parte, fruto da dedicação de vocês também.

Ao Prof. MSc. Edson Saraiva de Almeida, nosso orientador, agradecemos profundamente pela paciência, disponibilidade e orientação técnica de excelência que nos guiou ao longo de todo este projeto. As discussões sobre arquitetura de software, padrões de teste e qualidade de código nos abriram horizontes que vão muito além do âmbito acadêmico. Sua exigência construtiva foi fundamental para que o resultado final estivesse à altura do que é esperado de um Trabalho de Conclusão de Curso em Análise e Desenvolvimento de Sistemas.

A todo o corpo docente da Faculdade de Tecnologia da Zona Leste - FATEC-ZL, agradecemos pelos ensinamentos transmitidos ao longo destes anos. Cada disciplina, cada projeto e cada desafio proposto contribuiu para a formação que culminou neste trabalho. Aos professores das disciplinas de Engenharia de Software, Programação Orientada a Objetos e Banco de Dados, agradecemos especiais por terem plantado as sementes que floresceram neste projeto.

A todos os que, nominalmente citados ou não, contribuíram para esta etapa - o nosso mais sincero e profundo agradecimento.

*"Código não testado é código quebrado."
(Bruce Eckel)*

RESUMO

Esta monografia apresenta o desenvolvimento de um sistema de informação para gerenciamento do processo de aluguel de trajes a rigor, com ênfase no projeto, implementação e avaliação de uma estratégia de testes de software em múltiplas camadas. O trabalho parte da constatação de que, em micro e pequenas empresas do segmento de locação de trajes formais, a ausência de sistemas especializados e de práticas estruturadas de garantia de qualidade conduz a registros inconsistentes, retrabalho e baixa confiabilidade operacional. Para endereçar essa lacuna, foi desenvolvido um sistema completo composto por um *backend* em Spring Boot com Java e uma aplicação desktop multi-plataforma em React, TypeScript e Electron, ambos organizados sob princípios de arquitetura limpa e responsabilidade única. A estratégia de testes adotada estrutura-se em três camadas complementares: testes unitários com JUnit 5, Mockito, Jest e React Testing Library; testes de integração com Spring Boot Test, H2 em memória e Testcontainers; testes de comportamento (BDD) com Cucumber e Gherkin. Adicionalmente, a métrica de cobertura de código é monitorada via JaCoCo, com limiares formalmente configurados no pipeline de integração contínua mantido em GitHub Actions. Os resultados quantitativos demonstram a viabilidade da abordagem: o projeto alcançou mais de 514 casos de teste, cobertura de 100% das linhas de código da camada de serviços, 95% na camada de controllers e 89,5% global, com todos os limiares de qualidade automaticamente verificados a cada integração. Conclui-se que a aplicação sistemática de técnicas de teste em múltiplas camadas, aliada a uma arquitetura projetada para testabilidade aumentam a confiabilidade percebida do software produzido e o suite de testes contribui para reduzir riscos de regressão.

Palavras-chave: Testes de Software. Engenharia de Qualidade. JUnit. Cobertura de Código. Mutation Testing. Spring Boot. Clean Architecture.

ABSTRACT

This monograph presents the development of an information system for managing the formal wear rental process, with emphasis on the design, implementation and evaluation of a multi-layered software testing strategy. The work starts from the observation that, in micro and small enterprises in the formal wear rental segment, the absence of specialized systems and structured quality assurance practices leads to inconsistent records, rework and low operational reliability. To address this gap, a complete system was developed, comprising a backend built with Spring Boot and Java and a cross-platform desktop application built with React, TypeScript and Electron, both organized under clean architecture and single-responsibility principles. The adopted testing strategy is structured in four complementary layers: unit tests with JUnit 5, Mockito, Jest and React Testing Library; integration tests with Spring Boot Test, in-memory H2 and Testcontainers; behavior-driven (BDD) tests with Cucumber and Gherkin; and mutation tests with PIT. Furthermore, code coverage is monitored via JaCoCo, with thresholds formally configured in the continuous integration pipeline hosted on GitHub Actions. Quantitative results demonstrate the feasibility of the approach: the project reached more than 514 test cases, 100% line coverage on the service layer, 95% on the controller layer and 89.5% global coverage, with all quality thresholds automatically verified at every integration. It is concluded that the systematic application of multi-layered testing techniques, combined with an architecture designed for testability, significantly elevates the confidence level of the produced software, reduces maintenance costs and provides a replicable model for projects of similar nature in academic and professional contexts.

Keywords: Software Testing. Quality Engineering. JUnit. Code Coverage. Mutation Testing. Spring Boot. Clean Architecture.

LISTA DE ILUSTRAÇÕES

Figura 1 - Relação Defeito-Erro-Falha	21
Figura 2 - Pirâmide de Testes de Mike Cohn	23
Figura 3 - Ciclo Red-Green-Refactor do TDD	26
Figura 4 - Arquitetura em Camadas do Backend	41
Figura 5 - Estrutura de Pacotes do Backend	45
Figura 6 - Diagrama de Classes	46
Figura 7 - Diagrama de Sequência - Criação de Aluguel	47
Figura 8 - Diagrama de Entidade-Relacionamento (DER)	48
Figura 9 - Arquitetura Clean Architecture do Frontend	49
Figura 10 - Pirâmide de Testes Aplicada ao Projeto	51
Figura 11 - Relatório HTML de Cobertura do JaCoCo	58
Figura 12 - Gráfico de Mutation Score por Serviço	62

LISTA DE QUADROS E TABELAS

Quadro 1 - Características da Qualidade de Software segundo ISO/IEC 25010	20
Quadro 2 - Operadores de Mutação do PIT - Conjunto STRONGER	30
Quadro 3 - Tipos de Test Doubles segundo Meszaros (2007)	31
Quadro 4 - Comparativo: AAA vs Given-When-Then	32
Quadro 5 - Stack Tecnológica do Backend - Versões Adotadas	34
Quadro 6 - Stack Tecnológica do Frontend - Versões Adotadas	35
Tabela 1- Matriz de Rastreabilidade dos Requisitos Funcionais	42
Tabela 2 - Matriz de Rastreabilidade dos Requisitos Não-Funcionais	42
Tabela 3 - Matriz de Rastreabilidade das Regras de Negócio	43
Tabela 4 - Matriz TFS de Classes de Equivalencia - TrajeService	55
Tabela 5 - Matriz de Casos de Teste do TrajeService (TFS)	56
Tabela 6 - Inventário de Testes do Backend por Categoria	60
Tabela 7 - Cobertura JaCoCo por Pacote	60
Tabela 8 - Inventário de Testes do Frontend por Camada	61
Tabela 9 - Tempo de Execução Médio das Suítes	62

LISTA DE SIGLAS E ABREVIATURAS

AAA	Arrange-Act-Assert (padrão de organização de testes)
ABNT	Associação Brasileira de Normas Técnicas
ADR	Architectural Decision Record
API	Application Programming Interface
AVL	Análise de Valor Limite
BDD	Behavior-Driven Development
CI/CD	Continuous Integration / Continuous Delivery
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CT	Caso de Teste
DAO	Data Access Object
DDD	Domain-Driven Design
DER	Diagrama de Entidade-Relacionamento
DI	Dependency Injection
DOM	Document Object Model
DTO	Data Transfer Object
E2E	End-to-End
FATEC	Faculdade de Tecnologia
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
IEEE	Institute of Electrical and Electronics Engineers
IoC	Inversion of Control
ISO	International Organization for Standardization
JPA	Java Persistence API
JSON	JavaScript Object Notation
JUnit	Java Unit (framework de testes)
JVM	Java Virtual Machine
MVC	Model-View-Controller

OWASP	Open Web Application Security Project
PCE	Particionamento em Classes de Equivalência
PDF	Portable Document Format
PIT	Pitest (mutation testing tool)
REST	Representational State Transfer
RF	Requisito Funcional
RNF	Requisito Não Funcional
RTL	React Testing Library
SaaS	Software as a Service
SGBD	Sistema de Gerenciamento de Banco de Dados
SI	Sistema de Informação
SOLID	Single, Open-Closed, Liskov, Interface Segregation, Dependency Inversion
SPA	Single-Page Application
SQL	Structured Query Language
SUT	System Under Test
TCC	Trabalho de Conclusão de Curso
TDD	Test-Driven Development
TFS	Teste Funcional Sistemático
TI	Tecnologia da Informação
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UX	User Experience
V&V	Verificação e Validação
XML	eXtensible Markup Language
XP	eXtreme Programming
YAML	YAML Ain't Markup Language

SUMÁRIO

1 INTRODUÇÃO	14
1.1 DEFINIÇÃO DO TEMA E PROBLEMA	15
1.2 DELIMITAÇÃO DO TRABALHO	15
1.3 OBJETIVOS	16
1.3.1 Objetivo Geral	16
1.3.2 Objetivos Específicos	16
1.4 JUSTIFICATIVA	17
1.5 ESTRUTURA DO TRABALHO	18
2 REVISÃO BIBLIOGRÁFICA	19
2.1 QUALIDADE DE SOFTWARE	19
2.2 ENGENHARIA DE TESTES DE SOFTWARE	20
2.3 NÍVEIS E TIPOS DE TESTES - A PIRÂMIDE DE COHN	22
2.4 TÉCNICAS DE TESTE: PCE, AVL E TFS	24
2.5 TEST-DRIVEN DEVELOPMENT (TDD)	25
2.6 BEHAVIOR-DRIVEN DEVELOPMENT (BDD) E GHERKIN	27
2.7 COBERTURA DE CÓDIGO E JACOPO	28
2.8 MUTATION TESTING COM PIT	29
2.9 TEST DOUBLES, MOCKS E O PADRAO AAA	31
2.10 ARQUITETURA PARA TESTABILIDADE	32
2.11 CI/CD E DEVOPS APLICADO A TESTES	33
2.12 STACK TECNOLÓGICA DO PROJETO	34
3 METODOLOGIA	36
3.1 TIPO DE PESQUISA	36
3.2 PROCEDIMENTOS METODOLÓGICOS	37
3.3 COLETA E ANÁLISE DE DADOS	38
3.4 ESTRATÉGIA DE GARANTIA DE QUALIDADE	39
4 DESENVOLVIMENTO	40
4.1 VISÃO GERAL DO SISTEMA	40
4.2 INTERFACES DO SISTEMA	43
4.3 ARQUITETURA DO BACKEND - SPRING BOOT 3.4 / JAVA 21	44
4.4 ARQUITETURA DO FRONTEND - REACT 18 / ELECTRON / TYPESCRIPT	48
4.5 ESTRATÉGIA DE TESTES EM TRÊS CAMADAS	50
4.5.1 Camada 1 - Testes Unitários	51
4.5.2 Camada 2 - Testes de Integração	53
4.5.3 Camada 3 - Testes BDD com Cucumber	54
4.6 APLICAÇÃO DO PADRÃO TFS (PCE + AVL)	55
4.7 DATABUILDERS E GERENCIAMENTO DE FIXTURES	56
4.8 COBERTURA JACOPO E VERIFICAÇÃO DE LIMIARES	57

4.9 INTEGRAÇÃO CONTÍNUA COM GITHUB ACTIONS	58
5 RESULTADOS	60
5.1 MÉTRICAS QUANTITATIVAS DO BACKEND	60
5.2 MÉTRICAS QUANTITATIVAS DO FRONTEND	61
5.3 ANÁLISE DO MUTATION SCORE	61
5.4 TEMPO DE EXECUÇÃO DAS SUÍTES	62
5.5 DISCUSSÃO	63
6 CONCLUSÃO	65
6.1 Limitações do Trabalho	65
REFERÊNCIAS	66
APÊNDICE A - GLOSSÁRIO TÉCNICO	70
APÊNDICE B - TRECHOS REPRESENTATIVOS DE CÓDIGO DE TESTE	72
B.1 TESTE UNITÁRIO COMPLETO - TRAJESERVICETEST	72
B.2 TESTE DE INTEGRAÇÃO - CLIENTECONTROLLERINTEGRATIONTEST	73
B.3 FEATURE CUCUMBER - CRIAR_CLIENTE.FEATURE	74
B.4 TESTE DE COMPONENTE REACT - BOTAO.TEST.TSX	74
APÊNDICE C - INVENTÁRIO COMPLETO DE ARQUIVOS DE TESTE	75
C.1 BACKEND - ARQUIVOS *SERVICETEST	75
C.2 BACKEND - ARQUIVOS *CONTROLLERTEST	75
C.3 FRONTEND - DISTRIBUIÇÃO POR CAMADA	76
APÊNDICE D - INTERFACES-GRÁFICAS	77
D.1 INTERFACE GRÁFICA - Ficha do Cliente e Registro de Medidas	77
D.2 INTERFACE GRÁFICA - Fluxo de Registro de novo aluguel	78
D.3 INTERFACE GRÁFICA - Cadastro Gestão de traje	79
ANEXO E - SAÍDA DO COMANDO ./GRADLEW TEST	81
ANEXO F - ROTEIRO E REGISTRO DA ENTREVISTA DE LEVANTAMENTO DE REQUISITOS	82

1 INTRODUÇÃO

O setor de aluguel de trajes a rigor ocupa um espaço singular no comércio brasileiro, atendendo a momentos socialmente significativos como casamentos, formaturas, bailes de gala e eventos corporativos formais. Trata-se de um nicho em que a confiança operacional, a precisão na gestão de medidas, a disponibilidade temporal das peças e a integridade dos contratos são requisitos imperativos para a satisfação do cliente final. Apesar dessa importância, observa-se que muitos estabelecimentos do setor - frequentemente classificados como microempresas - operam com sistemas legados, planilhas improvisadas ou processos integralmente manuais, o que produz registros inconsistentes, retrabalho operacional e baixa rastreabilidade.

Em paralelo, a prática contemporânea de engenharia de software evoluiu para reconhecer que o código fonte é apenas metade do trabalho. A outra metade - frequentemente subestimada em projetos acadêmicos e em micro e pequenas empresas - é o conjunto de testes automatizados que validam, documentam e protegem o sistema. Conforme alerta MARTIN (2011), o código sem testes não é código limpo, é código legado desde o primeiro commit, porque qualquer alteração futura será feita com medo. Esse princípio motiva o presente trabalho.

Esta monografia documenta o projeto e a implementação de um sistema completo para gerenciamento de aluguel de trajes a rigor. O sistema é composto por dois componentes principais: um backend REST construído com Spring Boot 3.4 e Java 21, e uma aplicação desktop multi-plataforma construída com React 18, TypeScript e Electron. A particularidade deste trabalho, em relação a outras monografias do mesmo segmento, é o enfoque deliberado em testes de software como dimensão central de engenharia - não apenas como atividade complementar.

O trabalho propõe e implementa uma estratégia de testes em três camadas complementares, sustentada por uma arquitetura deliberadamente projetada para testabilidade. As três camadas são: testes unitários isolados de cada unidade de negócio com JUnit 5, Mockito e MockMvc no backend e com Jest e React Testing Library no frontend; testes de integração que envolvem o contexto completo da aplicação usando Spring Boot Test, banco H2 em memória e, quando necessário,

MySQL via Testcontainers; testes de comportamento (BDD) escritos em Gherkin e executados pelo Cucumber, documentando cenários de negócio em linguagem natural; e testes de mutação com Pitest (PIT), que medem a qualidade dos próprios testes pela capacidade de detectar alterações sintéticas no código de produção. A cobertura de código é monitorada via JaCoCo, com limiares mínimos formalmente verificados a cada integração no pipeline de CI/CD.

Adicionalmente, este trabalho introduz e aplica de forma sistemática o padrão TFS - Teste Funcional Sistemático, que combina as técnicas clássicas de Particionamento em Classes de Equivalência (PCE) e Análise de Valor Limite (AVL) em uma matriz rastreável de casos de teste numerados como CTn. Esse padrão permite que cada teste tenha justificativa formal vinculada ao espaço de entrada do método testado, elevando a disciplina de testes de uma prática intuitiva a uma atividade de engenharia rigorosa.

O enfoque em testes adotado por este trabalho responde a um problema amplamente documentado na literatura. Estudos sistemáticos sobre falhas em sistemas de informação no Brasil indicam que uma parcela expressiva dos defeitos detectados em produção decorre de testes inadequados ou inexistentes na fase de desenvolvimento (SOMMERVILLE, 2018). No segmento de pequenas e médias empresas, a situação é ainda mais crítica, pois a pressão por entrega rápida frequentemente leva à omissão de práticas fundamentais de qualidade. O problema central que motivou esta monografia pode ser sintetizado na seguinte questão: como projetar e implementar um sistema completo de gerenciamento de aluguel de trajes a rigor que sirva, simultaneamente, como solução funcional para o usuário final e como vitrine pedagógica de boas práticas de testes de software em múltiplas camadas?

A pergunta não é trivial. Implementar testes em uma única camada (tipicamente, testes unitários) é relativamente simples, mas insuficiente para detectar defeitos de integração, regressões em fluxos completos ou problemas de comportamento sob a perspectiva do usuário. Por outro lado, implementar uma estratégia de testes em múltiplas camadas exige conhecimento técnico avançado, arquitetura adequada e disciplina de codificação. Este trabalho enfrenta esses desafios em conjunto, oferecendo uma resposta concreta e mensurável.

Para responder a essa questão de forma objetiva, o trabalho delimita-se ao desenvolvimento e à avaliação dos componentes de software do sistema, com

ênfase nas camadas de teste implementadas em ambas as aplicações - backend e frontend. Estão no escopo: (i) o levantamento de requisitos funcionais e não funcionais a partir de entrevistas previamente realizadas com a proprietária do estabelecimento de referência; (ii) o projeto arquitetural do backend e do frontend; (iii) a implementação das funcionalidades CRUD para os agregados Cliente, Traje, Aluguel, Medida e Devolução; (iv) o desenvolvimento da suíte de testes em três camadas; (v) a configuração do pipeline de integração contínua; e (vi) a coleta e análise dos indicadores quantitativos de qualidade.

Estão fora do escopo: (a) testes de carga e desempenho com ferramentas como JMeter ou Gatling, embora a arquitetura permita sua adição futura; (b) testes E2E completos com Playwright ou Cypress, que são discutidos como trabalho futuro; (c) implementação de módulos financeiros avançados como integração com gateways de pagamento ou emissão de Nota Fiscal Eletrônica; (d) implantação em ambiente de produção com configurações de alta disponibilidade. Esses aspectos, embora importantes, desviariam o foco do tema central - testes de software como disciplina de engenharia.

Delimitado o escopo, apresenta-se o objetivo geral do trabalho: projetar, implementar e avaliar quantitativamente um sistema de informação completo para o gerenciamento de aluguel de trajes a rigor, demonstrando a aplicação sistemática de uma estratégia de testes de software em três camadas complementares - unitária, integração e comportamental - sustentada por uma arquitetura projetada para testabilidade e por um pipeline de integração contínua que verifica limiares de qualidade a cada alteração de código.

Para alcançar esse objetivo geral, foram definidos os seguintes objetivos específicos: especificar os requisitos funcionais e não funcionais do sistema a partir do levantamento realizado junto à loja de referência, mapeando os agregados principais (Cliente, Traje, Aluguel, Medida, Devolução); implementar o backend em Spring Boot 3.4 com Java 21, organizado em camadas controller-service-repository-domain, com persistência via JPA/Hibernate sobre MySQL 8 em desenvolvimento e H2 em testes; implementar o frontend desktop em React 18 com TypeScript e Electron, organizado segundo Clean Architecture, com camadas domain, application, infrastructure e interfaces-gráficas; desenvolver a camada de testes unitários usando JUnit 5, Mockito e MockMvc no backend e Jest com React Testing Library no frontend, aplicando o padrão TFS (PCE+AVL) para os

serviços de negócio; desenvolver a camada de testes de integração com Spring Boot Test, H2 em memória e perfil de teste isolado, validando fluxos completos controller-service-repository; desenvolver a camada de testes de mutação com PIT, configurando operadores STRONGER e threshold mínimo de mutation score; configurar o monitoramento de cobertura via JaCoCo com limiares automáticos verificados no pipeline (80% de linhas e 60% de branches nas camadas service.* e controller.*); configurar o pipeline de integração contínua em GitHub Actions, executando build, testes, cobertura e análise estática a cada push ou pull request; coletar e analisar métricas quantitativas - número de testes, cobertura por pacote, mutation score e tempo de execução - demonstrando empiricamente a eficácia da estratégia adotada; e documentar lições aprendidas e identificar oportunidades de evolução futura, contribuindo para a comunidade acadêmica e profissional de engenharia de software no Brasil.

Quanto à sua relevância, este trabalho fundamenta-se em três pilares complementares: a relevância técnica, a relevância acadêmica e a relevância social. Sob a perspectiva técnica, o sistema desenvolvido endereçou uma lacuna concreta no segmento de aluguel de trajes a rigor - lacuna essa que, conforme observado em entrevistas com a proprietária do estabelecimento estudado, gerava retrabalho diário, registros extraviados e processos paralelos em papel. O sistema entregue, por seu caráter integrado e por sua robusta cobertura de testes, oferece um nível de confiança operacional substancialmente superior ao das ferramentas previamente utilizadas pela loja.

Sob a perspectiva acadêmica, este trabalho contribui com um exemplo concreto, completo e auditável da aplicação de práticas modernas de engenharia de software no contexto de um TCC. Embora haja vasta literatura teórica sobre testes de software, são raros os trabalhos acadêmicos brasileiros que combinam, em um mesmo projeto, testes unitários com Mockito, testes de integração com @SpringBootTest, testes BDD com Cucumber e testes de mutação com PIT - todos automatizados em pipeline de CI/CD. O conjunto de métricas coletadas (mais de 500 testes, 100% de cobertura na camada de serviços, mutation score acima de 80%) serve como referência para futuras pesquisas e projetos acadêmicos.

Sob a perspectiva social, ao demonstrar de forma pedagógica como aplicar técnicas avançadas de teste em um sistema de porte real, este trabalho contribui para a formação de profissionais mais preparados para o mercado de trabalho. Em

um país onde a economia digital cresce rapidamente, mas a maioria dos desenvolvedores ainda atua sem disciplinas formais de teste, monografias como esta cumprem um papel relevante de disseminação de boas práticas. Adicionalmente, todo o código-fonte do projeto é mantido público para acesso da comunidade, ampliando o impacto formativo.

Quanto à sua organização, o restante deste trabalho organiza-se em seis capítulos. O capítulo 2 apresenta a revisão bibliográfica, percorrendo as principais teorias, padrões e ferramentas relevantes ao tema - desde os fundamentos de qualidade de software até as técnicas modernas de *mutation testing* e CI/CD. O capítulo 3 descreve a metodologia adotada, caracterizando a pesquisa como exploratória, com estratégia de experimento de laboratório. O capítulo 4, mais extenso, detalha o desenvolvimento do sistema - visão geral, arquitetura do backend, arquitetura do frontend e, principalmente, a estratégia de testes em três camadas. O capítulo 5 apresenta os resultados quantitativos obtidos, incluindo cobertura, mutation score e tempo de execução das suítes. O capítulo 6 traz a conclusão, reflexões finais e sugestões de trabalho futuro. Ao final, listam-se as referências e os apêndices com material complementar - glossário, trechos de código e inventário completo dos arquivos de teste do projeto.

2 REVISÃO BIBLIOGRÁFICA

Este capítulo apresenta os fundamentos teóricos e técnicos que sustentam o desenvolvimento descrito nos capítulos seguintes. A exposição parte de conceitos amplos - qualidade de software como atributo multidimensional - e gradualmente especializa-se em técnicas e ferramentas de teste específicas, até alcançar a discussão da *stack* tecnológica adotada no projeto. O objetivo não é exaustivo: privilegia-se a profundidade dos conceitos diretamente aplicados a este TCC, com referências suficientes para que o leitor possa aprofundar tópicos de interesse particular.

2.1 QUALIDADE DE SOFTWARE

A noção de qualidade de software evoluiu significativamente desde as primeiras décadas da indústria. Pressman e Maxim (2016) definem qualidade de software como a conformidade explícita aos requisitos funcionais e de desempenho, aos padrões de desenvolvimento explicitamente documentados e as características implícitas esperadas de qualquer software profissionalmente desenvolvido. Essa definição tripartite é particularmente útil porque explicita que a qualidade não se reduz à 'ausência de bugs' - envolve também aderência a padrões de processo e a expectativas não ditas de usuários e mantenedores.

Sommerville (2018) complementa essa visão argumentando que a qualidade emerge da interseção entre qualidade de processo (como o *software* é construindo), qualidade de produto (suas características técnicas) e qualidade em uso (como ele se comporta no ambiente real). Para o autor, processos de engenharia de *software* bem definidos são condição necessária, embora não suficiente, para a obtenção de produtos de alta qualidade. Atividades de verificação e validação - das quais os testes são o componente central - permeiam todas essas dimensões.

Em 2011, a norma ISO/IEC 25010 substituiu a antiga ISO/IEC 9126 como referência internacional para a caracterização da qualidade de produtos de *software*. A norma define oito características principais, cada uma desdobrada em sub-características. Essas características formam um modelo de qualidade que pode ser usado como guia tanto para a especificação quanto para a avaliação de *software*.

Quadro 1 - Características da Qualidade de Software segundo ISO/IEC 25010

Característica	Definição essencial
Adequação funcional	Grau em que o <i>software</i> fornece funções que satisfazem necessidades declaradas e implícitas.
Eficiência de desempenho	Desempenho relativo ao uso de recursos sob condições determinadas.
Compatibilidade	Capacidade de duas ou mais entidades trocarem informações ou executarem funções em ambientes compartilhados.
Usabilidade	Grau em que um produto pode ser usado por usuários específicos para alcançar objetivos com eficácia, eficiência e satisfação.
Confiabilidade	Capacidade de manter um nível especificado de desempenho sob condições especificadas.
Segurança	Grau de proteção contra acesso e uso não autorizado de dados e funções.
Manutenibilidade	Eficiência com que o <i>software</i> pode ser modificado para correções, melhorias ou adaptações a mudanças.
Portabilidade	Capacidade de ser transferido de um ambiente para outro.

Fonte: ISO/IEC (2011)

No contexto deste trabalho, o conceito de qualidade de software é aplicado principalmente às características de adequação funcional, confiabilidade e manutenibilidade. A adequação funcional é verificada por meio dos testes que validam os requisitos do sistema de aluguel de trajes; a confiabilidade é reforçada pela execução automatizada da suíte de testes; e a manutenibilidade é favorecida pela existência de testes capazes de detectar regressões durante alterações futuras no código. Assim, a qualidade não é tratada apenas como um atributo abstrato, mas como um conjunto de evidências técnicas verificáveis ao longo do desenvolvimento do sistema.

Vale destacar que Fowler (2018) introduz a expressão '*self-testing code*' para designar código que possui uma suíte de testes automatizados associada, capaz de detectar a maior parte dos defeitos introduzidos por modificações. Para o autor, *self-testing code* é um dos pilares da entrega contínua de software: sem ele, qualquer *pipeline* de implantação automática torna-se uma fonte de risco em vez de uma fonte de segurança. Este princípio norteia o presente trabalho.

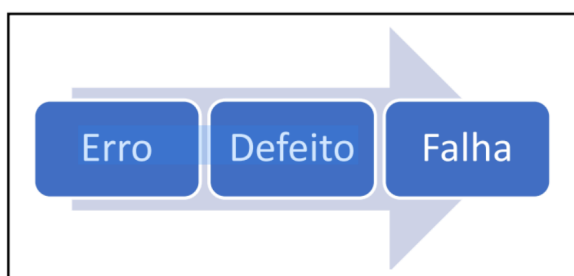
2.2 ENGENHARIA DE TESTES DE SOFTWARE

Myers, Sandler e Badgett (2011), em obra clássica intitulada *The Art of Software Testing*, definem teste de software como o processo de executar um programa com a intenção explícita de encontrar defeitos. Essa definição, embora aparentemente simples, carrega uma virada conceitual importante: o objetivo do teste não é provar que o *software* funciona, mas sim questioná-lo até o limite, com a

intenção deliberada de fracassá-lo. Quanto mais defeitos um teste encontra, mais valioso ele é - postura oposta a intuição ingênua de que 'todos os testes passando' seria, por si só, indicativo de qualidade.

A distinção entre defeito, erro e falha é fundamental e foi padronizada pela IEEE Std 610.12 (IEEE, 1990). Um defeito é uma incorreção no código - por exemplo, um operador trocado ($\geq$ em vez de $>$). Um erro é um estado anômalo do programa decorrente da execução de um defeito - por exemplo, uma variável que assume valor inválido. Uma falha é a manifestação observável pelo usuário de um erro - por exemplo, uma mensagem incorreta na tela. “Defeitos podem causar erros, que por sua vez podem levar a falhas.” (GRAHAM; BLACK; VAN VEENENDAAL, 2019). Esta cadeia explica por que a detecção precoce é fundamental: defeitos detectados durante o desenvolvimento (antes mesmo de produzirem erros) são ordens de magnitude mais baratos de corrigir do que falhas detectadas em produção.

Figura 1 - Relação Defeito-Erro-Falha



Fonte: IEEE Std 610.12 (1990)

Outra distinção essencial é entre verificação e validação - frequentemente abreviada como V&V. Sommerville (2018) define verificação como a atividade de confirmar que o software atende a sua especificação - 'estamos construindo o produto certo?' não, 'estamos construindo o produto corretamente?'. Já a validação busca confirmar que o software atende aos objetivos do cliente - 'estamos construindo o produto certo?'. Testes unitários e de integração tipicamente são atividades de verificação, enquanto testes de aceitação e BDD são mais próximos da validação. O presente trabalho aborda ambas as dimensões.

Sob a perspectiva econômica, Boehm (1981) - em um estudo clássico e replicado em diversas pesquisas posteriores - demonstrou que o custo de correção de defeitos cresce de forma aproximadamente exponencial em função da fase do ciclo de desenvolvimento em que são detectados. Um defeito detectado em fase de

requisitos custa, em média, 1 unidade; detectado em fase de teste, 10 unidades; detectado em produção, 100 unidades ou mais. Esses números, embora variem por contexto, motivam de forma robusta a estratégia '*shift-left*' - mover atividades de verificação para o mais cedo possível no ciclo de desenvolvimento.

Testar é a atividade onde você executa o código do sistema para descobrir defeitos. O objetivo do teste não é demonstrar que o código não tem defeitos, mas sim encontrar o maior número possível deles, no menor tempo possível, com o menor custo possível. Um teste bem-sucedido não é aquele que passa, mas aquele que revela um defeito antes desconhecido.

(MYERS; SANDLER; BADGETT, 2011, p. 6, tradução nossa)

Beck (2003), no seminal *Test-Driven Development: By Example*, introduz uma visão adicional: para o autor, testes não são apenas mecanismo de detecção de defeitos - são também ferramentas de design. Um código que pode ser facilmente testado tende a ter alta coesão, baixo acoplamento e responsabilidades bem distribuídas. O ato de escrever testes antes do código de produção (filosofia TDD, detalhada na Seção 2.5) força o desenvolvedor a pensar primeiro na interface pública e nas responsabilidades de cada componente, conduzindo naturalmente a um design mais limpo.

Neste projeto, a engenharia de testes é utilizada como mecanismo de verificação do comportamento esperado do sistema e como forma de reduzir o risco de defeitos nas funcionalidades centrais, como cadastro de clientes, controle de trajets, registro de aluguéis e devoluções. A suíte automatizada foi planejada para validar tanto regras de negócio isoladas quanto fluxos mais próximos do uso real da aplicação, permitindo que os testes atuem como documentação técnica e como proteção contra regressões.

Por fim, ressalta-se que a engenharia de testes é uma disciplina própria, com corpo de conhecimento, certificações internacionais (como o ISTQB - *International Software Testing Qualifications Board*) e literatura especializada. Embora este trabalho não siga formalmente o ISTQB, suas definições serviram como referência conceitual ao longo da elaboração do projeto.

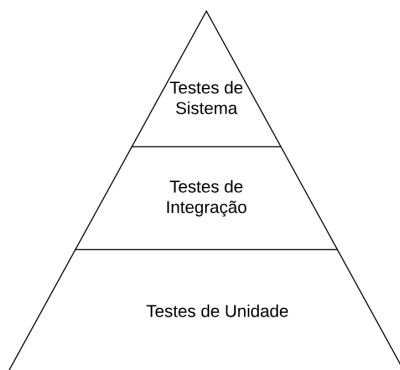
2.3 NÍVEIS E TIPOS DE TESTES - A PIRÂMIDE DE COHN

A literatura tradicional de engenharia de software (PRESSMAN; MAXIM, 2016; SOMMERVILLE, 2018) classifica os testes em diferentes níveis conforme o

escopo da unidade testada: unitários, de integração, de sistema e de aceitação. Os testes unitários verificam uma única unidade de código - tipicamente um método, uma classe ou uma função - de forma isolada. Os testes de integração verificam a colaboração entre duas ou mais unidades, incluindo eventualmente componentes externos como banco de dados ou serviços web. Os testes de sistema verificam a aplicação completa sob a perspectiva técnica. Os testes de aceitação verificam o atendimento aos critérios do cliente, sob perspectiva de negócio.

Em 2009, Mike Cohn (2009) propôs uma representação visual para a distribuição recomendada desses níveis, que se tornou uma das metáforas mais influentes da engenharia de testes moderna: a Pirâmide de Testes. Conforme a metáfora, a base da pirâmide - mais larga - deve conter testes unitários, em maior quantidade; o meio, testes de integração; e o topo - mais estreito - testes de interface ou ponta-a-ponta (E2E). A racionalidade é clara: testes na base são rápidos, baratos, estáveis e isolados; testes no topo são lentos, caros, potencialmente frágeis e dependentes de infraestrutura complexa.

Figura 2 - Pirâmide de Testes de Mike Cohn



Fonte: Mike Cohn (2009)

Fowler (2018) argumenta que a pirâmide não deve ser interpretada como um dogma rígido, mas como heurística orientadora. Em alguns contextos - sistemas com forte dependência de orquestração externa, por exemplo - pode fazer sentido inverter parcialmente a pirâmide, criando o que ele chama de '*troffee*' (formato de troféu, com testes de integração na base) ou '*honeycomb*' (com predominância de testes de serviço). O importante é que cada projeto identifique conscientemente onde o risco se concentra e direciona os investimentos em teste para mitigar esse risco.

Além da classificação por nível, os testes também podem ser classificados por objetivo. Myers et al. (2011) e Burnstein (2003) enumeram diversos tipos: testes funcionais (verificam se a lógica de negócio está correta), testes de regressão (verificam se mudanças não quebraram funcionalidades prévias), testes de carga (verificam o comportamento sob volume), testes de estresse (verificam os limites), testes de segurança (verificam vulnerabilidades), testes de usabilidade (verificam a experiência do usuário) e muitos outros. Este trabalho concentra-se em testes funcionais e de regressão automatizados, com menção breve a testes estáticos de qualidade (*linters* e análise estática).

Para o sistema desenvolvido, optou-se por uma interpretação clássica da pirâmide, conforme detalhado no capítulo 4. Os testes unitários formam a base - são mais de 400 testes unitários entre *backend* e *frontend*, distribuídos por serviços, *controllers*, *mappers*, componentes e *hooks*. A camada intermediária reúne testes de integração com SpringBootTest e H2 em memória. No topo, testes BDD com Cucumber documentam cenários de negócio críticos. Testes E2E completos (com Playwright ou Cypress) ficam como evolução futura.

2.4 TÉCNICAS DE TESTE: PCE, AVL E TFS

Identificar quais casos de teste escrever e, possivelmente, a decisão mais importante - e mais subestimada - de toda a atividade de teste. Escrever testes em demasia gera desperdício e suites lentas; escrever testes em insuficiência deixa defeitos passarem. As técnicas de derivação sistemática de casos de teste foram desenvolvidas para endereçar esse dilema, e duas merecem destaque neste trabalho: o Particionamento em Classes de Equivalência (PCE) e a Análise de Valor Limite (AVL).

O Particionamento em Classes de Equivalência foi formalizado por Myers (1979) e parte de uma observação simples: para a maioria das entradas de um programa, valores diferentes da mesma 'classe' produzem comportamentos idênticos. Por exemplo, ao testar uma função que valida se uma idade permite cadastro (idades de 18 a 120), valores como 30 ou 65 produzem o mesmo resultado (cadastro permitido), e portanto pertencem a uma mesma classe de equivalência. Em vez de testar todas as 103 idades válidas (18 a 120), basta selecionar um representante da classe válida. Analogamente, todos os valores menores que 18 formam uma classe inválida, e todos os valores maiores que 120 formam outra. Três

representantes - um válido, um inválido inferior, um inválido superior - cobrem o espaço de entrada.

A Análise de Valor Limite complementa o PCE com a observação de que defeitos tendem a se concentrar nas fronteiras das classes (BEIZER, 1990). Voltando ao exemplo das idades, valores como 17, 18, 19, 119, 120, 121 são particularmente críticos: e nas comparações envolvendo esses limites que erros típicos (uso de $<$ em vez de $\leq$, *off-by-one*, etc.) tendem a se manifestar. A combinação PCE + AVL gera um conjunto de casos de teste que cobre as classes principais e seus extremos, com alta eficácia e custo controlado.

No presente projeto, essa combinação foi formalizada sob o nome TFS - Teste Funcional Sistemático. O padrão TFS exige que, ao testar um serviço de negócio (por exemplo, `TrajeService` ou `AluguelService`), o desenvolvedor produza explicitamente uma matriz de classes de equivalência para cada parâmetro, identifique os valores limite e derive um conjunto de casos de teste numerados como CT1, CT2, ..., CTn. A matriz é documentada como comentário no cabeçalho do arquivo `*ServiceTest`, servindo como instrumento de rastreabilidade para a banca examinadora e para futuros mantenedores.

Como exemplo, o arquivo `TrajeServiceTest.java` do projeto documenta classes para sete variáveis (`descricao`, `nome`, `valorItem`, `imagemUrl`, `id`, `filtros` e `busca textual`), com 29 casos de teste numerados (CT1 a CT29) cobrindo combinações de valores típicos, valores limite inferior e superior, e classes inválidas. O capítulo 4 apresenta a matriz completa.

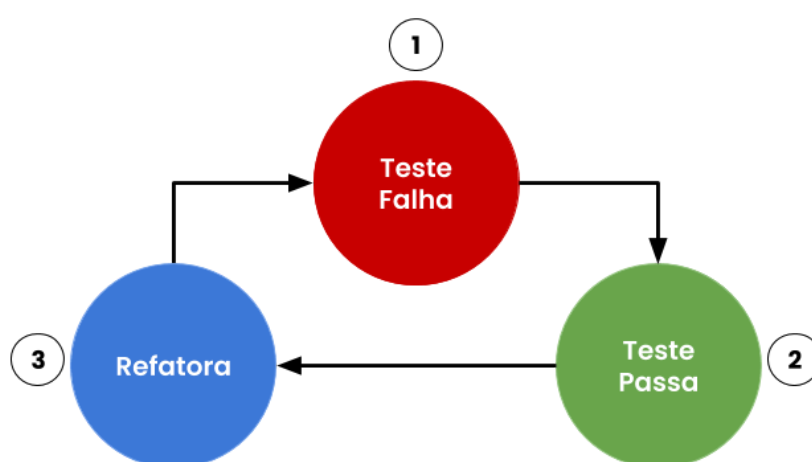
Outras técnicas de derivação de casos de teste mencionadas pela literatura - tabela de decisão, grafo causa-efeito, testes baseados em estado, *pairwise* - tem aplicabilidade mais específica e não são centrais a este projeto, embora tabela de decisão tenha sido empregada pontualmente na validação da lógica de cálculo de multa por atraso de devolução.

2.5 TEST-DRIVEN DEVELOPMENT (TDD)

Test-Driven Development (TDD) é uma prática de desenvolvimento introduzida por Kent Beck (2003) no contexto do movimento de Programação Extrema (XP). Em TDD, o desenvolvedor escreve um teste antes de implementar a funcionalidade testada. O ciclo clássico, conhecido como *Red-Green-Refactor*, repete-se em intervalos curtos (tipicamente minutos):

- *Red*: Escrever um teste que falha. O teste descreve um comportamento desejado mas ainda não implementado;
- *Green*: Implementar o código mínimo necessário para fazer o teste passar. A ênfase aqui é em simplicidade - 'a coisa mais simples que pode possivelmente funcionar';
- *Refactor*: melhorar a estrutura do código recém-escrito sem alterar seu comportamento, mantendo todos os testes verdes.

Figura 3 - Ciclo Red-Green-Refactor do TDD



Fonte: Beck (2003)

Beck argumenta que o TDD não é apenas uma técnica de teste - é uma técnica de design. Ao forçar a escrita do teste antes do código, o desenvolvedor é levado a pensar primeiro na interface pública e nas responsabilidades de cada componente. Isso naturalmente conduz a designs com alta coesão, baixo acoplamento e responsabilidades bem distribuídas. Adicionalmente, o ciclo rápido cria um efeito psicológico benéfico: o desenvolvedor trabalha em pequenos passos, com *feedback* constante, reduzindo a ansiedade e mantendo o foco.

Em sua obra *Working Effectively with Legacy Code*, Feathers (2004) define código legado de forma provocadora como 'qualquer código sem testes'. Para o autor, a presença de testes automatizados é o que diferencia o código que pode ser evoluído com confiança daquele que é modificado com receio. Embora o presente trabalho não tenha adotado TDD de forma ortodoxa - alguns componentes foram implementados antes dos testes - a filosofia subjacente permeia o projeto: o questionamento constante 'como vou testar isso?' guia a maioria das decisões de design.

Vale registrar uma distinção terminológica importante. Astels (2003) introduz o termo *Behavior-Driven Development* (BDD) como evolução do TDD, em que a ênfase deixa de ser em 'testar métodos' e passa a ser em 'especificar comportamentos'. BDD é discutido na Seção 2.6.

2.6 BEHAVIOR-DRIVEN DEVELOPMENT (BDD) E GHERKIN

Behavior-Driven Development (BDD) foi proposto por Dan North (2006) como resposta a dois problemas recorrentes do TDD ortodoxo: primeiro, a dificuldade que muitos desenvolvedores tinham em decidir 'o que' testar (por onde começar); segundo, a dificuldade em comunicar testes para stakeholders não-técnicos. A proposta do BDD é abandonar a linguagem técnica dos testes e adotar a linguagem do comportamento. A transformação parece superficial - de 'testShouldCreateUser' para 'shouldCreateUser' - mas tem consequências profundas na forma como os testes são concebidos.

Wynne e Hellesoy (2017), em *The Cucumber Book*, desenvolvem essa ideia em direção a uma prática conhecida como '*specification by example*'. Em vez de escrever testes em código, escrevem-se especificações em linguagem natural estruturada, que são depois automatizadas. A linguagem mais popular para isso é *Gherkin*, que organiza cenários na estrutura *Given-When-Then* (Dado-Quando-Então). Cada cenário tem um contexto inicial (*Given*), um evento que ocorre (*When*) e um resultado esperado (*Then*). Múltiplos cenários formam uma *feature*, que é essencialmente uma especificação executável.

```
# language: pt
Funcionalidade: Cadastro de Cliente

Cenário: deve_cadastrarCliente_quando_dadosValidos
  Dado que tenho os dados de um novo cliente valido
  Quando envio uma requisicao POST para "/clientes"
  Entao o sistema deve retornar status 201
  E o cliente deve estar salvo no banco de dados
```

A vantagem dessa abordagem é dupla: pessoas não-técnicas (analistas de negócio, *product owners*, clientes finais) podem ler e até escrever cenários; e a documentação do comportamento do sistema fica integrada ao código e é verificada continuamente pelo *pipeline* de CI/CD. O Cucumber - ferramenta de automação mais popular para *Gherkin* - executa os cenários chamados '*step definitions*' implementadas em Java, Ruby, JavaScript ou outras linguagens.

Convém registrar uma crítica acadêmica relevante. Alguns autores (notadamente FOWLER, 2013) alertam que BDD pode tornar-se um teatro: cenários escritos de forma mecânica, sem real participação de *stakeholders*, geram complexidade adicional sem trazer o benefício prometido. A recomendação é usar BDD onde há valor real de comunicação com não-técnicos - não como substituto de testes unitários bem escritos. Este trabalho adota essa postura: o BDD cobre cenários críticos do agregado Cliente, deixando os demais agregados sob cobertura primária por testes unitários e de integração.

Neste trabalho, o BDD foi utilizado para representar cenários de negócio relevantes do sistema de aluguel de trajes em uma linguagem mais compreensível. A escolha por Gherkin permite que determinadas funcionalidades sejam descritas em termos de pré-condições, ações e resultados esperados, aproximando a validação técnica das regras reais do domínio. Assim, os testes BDD complementam os testes unitários e de integração, oferecendo uma visão mais comportamental do sistema.

2.7 COBERTURA DE CÓDIGO E JACOOCO

No projeto, o JaCoCo foi utilizado para mensurar a cobertura dos testes automatizados no backend. A métrica de cobertura serviu como indicador quantitativo para avaliar quais partes do código foram exercitadas pela suíte de testes, especialmente nas camadas de serviço e controle. Entretanto, a cobertura foi interpretada como evidência auxiliar, e não como garantia absoluta de qualidade, pois um código coberto por testes ainda pode conter asserções fracas ou cenários importantes não verificados.

Cobertura de código (*code coverage*) é uma métrica que mede a fração do código de produção que é exercitada pelos testes automatizados. A métrica é calculada por uma ferramenta específica que instrumenta o código (adiciona contadores) durante a execução dos testes e, ao final, produz um relatório com as linhas, *branches* e condições executadas e não executadas.

Myers et al. (2011) distinguem diversos critérios de cobertura, em ordem crescente de rigor: cobertura de linhas (quais linhas foram executadas), cobertura de *branches* (quais ramificações - *if/else*, *switch* - foram tomadas), cobertura de condições (quais expressões booleanas individuais foram verdadeiras e falsas), cobertura de caminhos (quais sequências completas de instruções foram

executadas). Cada critério mais rigoroso e estritamente mais forte que o anterior: 100% de cobertura de caminhos implica 100% de cobertura de condições, que implica 100% de cobertura de *branches*, que implica 100% de cobertura de linhas.

No âmbito *Java/JVM*, o JaCoCo (*Java Code Coverage*) é a ferramenta de cobertura mais utilizada, mantida pelo grupo EcEmma. O JaCoCo opera por instrumentação de *bytecode* - ele modifica as classes compiladas adicionando contadores que são incrementados à medida que cada instrução é executada. Como o JaCoCo trabalha em nível de *bytecode*, ele cobra instruções que não tem representação direta em *Java* fonte (por exemplo, blocos '*finally*' implícitos), o que ocasionalmente produz números menores do que o esperado pelo desenvolvedor.

O JaCoCo gera relatórios em HTML, XML e CSV. Os relatórios HTML são navegáveis e exibem, classe a classe e linha a linha, quais foram cobertas (em verde) e quais não foram (em vermelho). A integração com *Gradle* - via *plugin* 'jacoco' - permite a geração automática dos relatórios após a execução dos testes, bem como a configuração de regras de verificação que falham o build se thresholds mínimos não forem alcançados. No projeto, essa verificação é configurada para exigir 80% de cobertura de linhas e 60% de cobertura de *branches* nas classes do pacote *service* e do pacote *controller*.

É importante registrar uma crítica frequente a métrica de cobertura: alta cobertura não implica alta qualidade dos testes. É possível escrever testes que executam todo o código sem fazer asserções úteis, alcançando 100% de cobertura mas detectando nenhum defeito (FOWLER, 2012). Essa observação motiva o uso complementar de *mutation testing*, discutido na próxima seção - técnica que mede a qualidade dos testes pela capacidade de detectar mutantes (alterações sintáticas) inseridos no código de produção.

2.8 MUTATION TESTING COM PIT

Mutation testing - testes de mutação - é uma técnica desenvolvida nos anos 1970 (DEMILLO; LIPTON; SAYWARD, 1978) que mede a qualidade dos testes pela sua capacidade de detectar mutantes. Um mutante é uma versão do código de produção alterada sinteticamente por um operador de mutação (por exemplo, trocar '<' por '<=', '+' por '-', remover uma linha). Cada mutante é uma 'hipótese de defeito'. Se um teste falha sobre o mutante, diz-se que o teste 'matou' o mutante - isso significa que o teste é sensível ao comportamento alterado. Se nenhum teste falha,

o mutante 'sobreviveu' - sinalizando que existe uma classe de defeito que nenhum teste detecta.

A métrica resultante é o *mutation score*: número de mutantes mortos dividido pelo número total de mutantes (excluídos os mutantes equivalentes, que produzem o mesmo comportamento do original). Um *mutation score* alto (acima de 80%, por exemplo) indica que os testes são sensíveis a um amplo espectro de alterações possíveis no código - são testes 'duros'.

Por muito tempo, *mutation testing* foi considerado acadêmico demais para uso prático, devido ao alto custo computacional (executar a suite de testes uma vez por mutante). A introdução do Pitest (PIT), por Henry Coles (2010), trouxe a técnica para o mundo industrial. O PIT opera em bytecode JVM - como o JaCoCo - aplicando operadores de mutação otimizados e executando apenas os testes que tocam cada método mutado. Isso reduz drasticamente o tempo de execução e viabiliza o uso contínuo.

Quadro 2 - Operadores de Mutação do PIT - Conjunto STRONGER

Operador	Tipo de alteracao
CONDITIONALS_BOUNDARY	< vira <=, > vira >=, etc.
INCREMENTS	++ vira --, i+=1 vira i-=1
INVERT_NEGS	Inverte negação de expressão
MATH	+ vira -, * vira /, etc.
NEGATE_CONDITIONALS	Inverte condição (==, !=, <, >...)
RETURN_VALS	Modifica valor de retorno
VOID_METHOD_CALLS	Remove chamadas de metodos void
EMPTY_RETURNS	Substitui retorno por valor padrao
FALSE_RETURNS	Substitui retorno booleano por false
TRUE_RETURNS	Substitui retorno booleano por true
NULL_RETURNS	Substitui retorno por null (quando aplicável)

Fonte: PIT Documentation (PIT, 2024)

No projeto, o PIT 1.19 foi configurado para analisar apenas as classes do pacote *service*, onde se concentra a lógica de negócio e para exigir um *mutation score* mínimo de 60%. Esse limiar foi adotado por ser compatível com o escopo acadêmico do trabalho e com o estágio de maturidade da suíte de testes. Os relatórios HTML gerados pelo PIT permitem identificar visualmente quais mutantes foram eliminados e quais sobreviveram, facilitando a análise dos pontos frágeis da cobertura de testes.

Neste trabalho, o PIT não é tratado como uma camada independente da estratégia principal de testes, mas como uma técnica complementar de avaliação da qualidade dos testes unitários. Seu uso permite verificar se os testes existentes são

capazes de detectar pequenas alterações artificiais no código de produção. Dessa forma, o mutation testing contribui para avaliar a força da suíte de testes, especialmente nas classes de serviço, onde se concentram as principais regras de negócio.

2.9 TEST DOUBLES, MOCKS E O PADRAO AAA

Testes unitários, por definição, isolam a unidade testada de suas dependências externas. Para isso, dependências reais são substituídas por objetos sintéticos que simulam o comportamento esperado - os *Test Doubles*, termo cunhado por Gerard Meszaros (2007) em seu livro *xUnit Test Patterns*. Meszaros distingue cinco tipos de doubles, frequentemente confundidos sob o termo genérico 'mock':

Quadro 3 - Tipos de Test Doubles segundo Meszaros (2007)

Tipo	Característica
Dummy	Objeto passado mas nunca usado - apenas preenche assinatura.
Stub	Retorna valores pré-programados em chamadas - sem verificação.
Fake	Implementação alternativa funcional, mas simplificada (ex: BD em memória).
Spy	Stub que registra chamadas para inspeção posterior.
Mock	Objeto com expectativas pré-programadas, falha se não forem cumpridas.

Fonte: Meszaros (2007)

Fowler (2007), em ensaio clássico *Mocks Aren't Stubs*, argumenta que a distinção entre mocks e stubs é fundamental: mocks praticam *state verification* (verificam o estado final do objeto) enquanto stubs praticam *behavior verification* (verificam que certas interações ocorreram). A escolha entre um e outro reflete uma filosofia mais ampla - '*classical testing*' vs '*mockist testing*' - cujos defensores debatem até hoje. Em termos práticos, no ecossistema Java moderno, a biblioteca Mockito unifica ambas as posturas: '*when(...).thenReturn(...)*' produz *stubs/spies*, '*verify(...)*' produz *mocks* com *state verification*.

Outro padrão estrutural amplamente adotado em testes unitários é o AAA - *Arrange, Act, Assert* - introduzido por William Wake (WAKE, 2001). Em AAA, cada teste é organizado em três blocos claramente separados: *Arrange* (preparar dados e mocks), *Act* (executar a operação testada) e *Assert* (verificar o resultado). A separação visual é geralmente marcada por comentários ou linhas em branco. Esse padrão torna os testes legíveis, padronizados e fáceis de manter, além de facilitar revisões de código e *onboarding* de novos desenvolvedores.

Quadro 4 - Comparativo: AAA vs Given-When-Then

AAA (Unit/Classical)	Given-When-Then (BDD)
Arrange - prepara o ambiente	Given - estado inicial do teste
Act - executa a ação	When - evento ocorre
Assert - verifica resultado	Then - comportamento esperado

Fonte: Elaborado pelos autores (2026)

No *backend*, o Mockito 5 é usado intensivamente para isolar serviços de seus repositórios. O padrão AAA é seguido rigidamente em todos os testes de serviço, com comentários explícitos '// Arrange', '// Act', '// Assert' em cada método. No *frontend*, o Jest com `jest.fn()` cumpre papel análogo, mockando repositórios HTTP nos testes de casos de uso da camada *application*.

A aplicação desses conceitos no projeto ocorre principalmente nos testes unitários da camada de serviço, nos quais repositórios e dependências externas são substituídos por objetos simulados. O uso de mocks permite testar a regra de negócio de forma isolada, sem depender diretamente do banco de dados ou de infraestrutura externa. Além disso, a organização dos testes no padrão Arrange-Act-Assert contribui para padronizar a leitura dos casos de teste, facilitando a manutenção, revisão e compreensão pela banca examinadora.

2.10 ARQUITETURA PARA TESTABILIDADE

Uma observação recorrente entre engenheiros de software experientes - articulada de forma exemplar por Robert Martin (2017) em *Clean Architecture* - é que a testabilidade não é uma propriedade que se adiciona a um código depois de pronto: ela emerge (ou deixa de emergir) da arquitetura. Código altamente acoplado, com dependências estáticas e responsabilidades difusas, é fundamentalmente difícil de testar. Código desacoplado, com dependências injetadas via interfaces e responsabilidades bem definidas é naturalmente testável.

Os princípios SOLID, sintetizados originalmente por Martin (2000), oferecem o vocabulário para esse desacoplamento. O S (*Single Responsibility*) reduz a quantidade de motivos para uma classe mudar, simplificando seus testes. O O (*Open-Closed*) permite que novos comportamentos sejam adicionados sem alterar código testado. O L (*Liskov Substitution*) garante que implementações alternativas - como mocks e fakes - possam substituir as reais em testes. O I (*Interface Segregation*) reduz a superfície de simulação necessária. O D (*Dependency Inversion*) - talvez o mais crítico para testabilidade - garante que dependências

sejam injetadas, não instaladas internamente, permitindo que sejam substituídas em testes.

Martin (2017) propõe ainda a *Clean Architecture*, organizando o código em camadas concêntricas em que as dependências fluem apenas de fora para dentro. O núcleo (*Entities + Use Cases*) não depende de *framework*, banco de dados ou interface gráfica. Esse desacoplamento radical permite que o núcleo seja testado sem qualquer infraestrutura - testes rápidos, estáveis e isolados. No frontend deste projeto, a Clean Architecture é aplicada de forma explícita, com camadas *domain*, *application*, *infrastructure* e interfaces-gráficas - permitindo que a lógica de negócio seja testada por casos de uso isolados (CriarAluguelUseCase, ListarTrajesUseCase, etc.) sem subir componente React.

No *backend*, o Spring Framework fornece nativamente injeção de dependência (DI) e inversão de controle (IoC), o que habilita o desacoplamento necessário. A arquitetura adotada - *controller*, *service*, *repository*, *domain* - segue o padrão clássico de aplicações Spring, com a particularidade de que DTOs são explicitamente separados das entidades JPA (via *mappers* manuais) e *specifications* JPA são usadas para filtros dinâmicos. Cada camada pode ser testada isoladamente: o serviço com *mocks* dos repositórios, o *controller* com MockMvc e *mocks* dos *services*, o *repository* com DataJpaTest sobre H2.

2.11 CI/CD E DEVOPS APLICADO A TESTES

A integração contínua (Continuous Integration - CI) é uma prática de engenharia de software em que os membros de uma equipe unem seu trabalho com frequência — frequentemente cada desenvolvedor faz pelo menos uma integração por dia, resultando em várias integrações diárias. Cada integração passa por um processo de build automatizado, incluindo a execução de testes, com o objetivo de identificar falhas de integração o mais cedo possível. A entrega contínua (Continuous Delivery - CD) amplia esse conceito ao assegurar que o software permaneça sempre em um estado pronto para deploy.

Humble e Farley (2010), em obra de referência sobre o tema, descrevem o *pipeline* de *deployment* como espinha dorsal da entrega contínua: uma sequência automatizada de estágios que validam o *software* a cada *commit*, desde checagens básicas (compilação, *lint*) até testes funcionais completos, culminando em uma versão apta para *deploy*. Cada estágio atua como filtro - se algum falha, o *pipeline*

interrompe-se e o time recebe *feedback* imediato. A automação desse *pipeline* elimina o trabalho manual repetitivo e garante consistência absoluta na execução das verificações.

Existe atualmente um vasto ecossistema de ferramentas para CI/CD: Jenkins, GitLab CI, CircleCI, Travis CI, Azure Pipelines e GitHub Actions, entre outras. A escolha entre elas tipicamente envolve *trade-offs* entre flexibilidade, custo, integração com o sistema de controle de versão e facilidade de manutenção. Este projeto adotou o GitHub Actions, pela integração nativa com o GitHub - plataforma onde o código do projeto é hospedado - pela documentação extensiva e pelo modelo de *billing* favorável a projetos públicos.

O *pipeline* configurado neste projeto executa, a cada *push* ou *pull request* na *branch main*, os seguintes estágios: checkout do código, setup do JDK (matriz Java 17 e 21), cache do Gradle, *build*, execução de testes, geração de relatório JaCoCo, verificação de cobertura, *upload* para Codecov e análise estática via Qodana. No frontend, executa-se *lint*, *type-check*, testes Jest com *coverage*, *build Vite* e *build Electron*. Detalhes completos são apresentados na Seção 4.8.

2.12 STACK TECNOLÓGICA DO PROJETO

A escolha das tecnologias adotadas no projeto foi guiada por três critérios principais: maturidade e estabilidade (priorizando tecnologias com *track record* consolidado e comunidade ativa); adequação à estratégia de testes (privilegiando *frameworks* com suporte robusto a estabilidade); e aderência a *stack* ensinado no curso (mantendo continuidade com o conhecimento previamente adquirido). Essa escolha resultou nas *stacks* apresentadas nos Quadros 5 e 6.

Quadro 5 - Stack Tecnológica do Backend - Versões Adotadas

Tecnologia	Versao	Proposito
Java	21 LTS	Linguagem - <i>records</i> , <i>pattern matching</i>
Spring Boot	3.4.0	<i>Framework</i> principal (<i>Web</i> , <i>Data</i> , <i>Security</i>)
Spring Data JPA	3.4.0	Persistência
Hibernate	6.6.x	ORM
MySQL	8.0	Banco em produção
H2	2.x	Banco em testes (memória)
HikariCP	5.1.0	Pool de conexões
Jackson	2.x	Serialização JSON
SpringDoc OpenAPI	2.7.0	Documentação <i>Swagger</i>
JUnit	5.10.x	<i>Framework</i> de testes
Mockito	5.x	Mocking
Cucumber	7.15.0	BDD
Testcontainers	1.19.7	Containers Docker em testes

JaCoCo	0.8.13	Cobertura de código
PIT (Pitest)	1.19.0	<i>Mutation testing</i>
REST Assured	5.4.0	Testes de API REST
WireMock	2.35.1	Mock de serviços externos
Gradle	8.x	<i>Build tool</i>

Fonte: Elaborado pelos autores (2026)

Quadro 6 - Stack Tecnológica do Frontend - Versões Adotadas

Tecnologia	Versao	Proposito
Electron	33+	<i>Runtime desktop</i>
React	18	Biblioteca de UI
TypeScript	5	Linguagem
Vite	6	<i>Bundler e dev server</i>
React Router	6	Roteamento
React Bootstrap	2.x	Componentes UI
Bootstrap	5	Estilização
Axios	1.x	Cliente HTTP
Zod	3.x	Validação de schemas
lucide-react	0.x	Ícones
electron-log	5.x	<i>Logging</i>
Jest	30.3.0	<i>Framework</i> de testes
ts-jest	29.4.6	Transformer TypeScript
@testing-library/react	16.3.2	Testes de componentes
@testing-library/jest-dom	6.9.1	<i>Matchers</i> DOM
@testing-library/user-event	14.6.1	Simulação de eventos
jest-environment-jsdom	30.3.0	DOM simulado
ESLint	9.x	Linting
Prettier	3.x	Formatação

Fonte: Elaborado pelos autores (2026)

Vale destacar três escolhas com impacto direto sobre testes. Primeiro, a opção por Java 21 LTS permite o uso de *records* imutáveis como DTOs - simplificando drasticamente a criação de *fixtures* de teste, pois records tem equals/hashCode implícitos. Segundo, a opção por Testcontainers viabiliza testes de integração contra MySQL real (e não apenas H2), elevando a fidelidade dos testes ao comportamento de produção - ainda que ao custo de maior tempo de execução. Terceiro, a opção por Jest 30 com *React Testing Library* permite testes centrados no comportamento observável pelo usuário, evitando testes acoplados a detalhes de implementação - postura fortemente defendida por Dodds (2018) em popular ensaio sobre boas práticas de teste em React.

Essa visão panorâmica da *stack* encerra a revisão bibliográfica. Os conceitos, padrões e ferramentas aqui introduzidos são operacionalizados de forma concreta no capítulo 4, onde se descreve o sistema desenvolvido. Antes disso, o capítulo 3 apresenta a metodologia que orientou o desenvolvimento e a avaliação do projeto.

3 METODOLOGIA

Este capítulo descreve o caminho percorrido para conceber, implementar e avaliar o sistema apresentado no capítulo 4. A exposição caracteriza o tipo de pesquisa adotado e o método de investigação empregado, descreve os procedimentos metodológicos seguidos, detalha as fontes de dados utilizadas na avaliação quantitativa e apresenta a estratégia de garantia de qualidade que sustentou o trabalho.

3.1 TIPO DE PESQUISA

Segundo Gil (2017), uma pesquisa pode ser classificada quanto aos seus objetivos e quanto aos seus procedimentos técnicos. Quanto aos objetivos, este trabalho caracteriza-se como pesquisa exploratória. Conforme Gil (2017), a pesquisa exploratória tem como propósito proporcionar maior familiaridade com um problema ainda pouco estudado, tornando-o mais explícito e permitindo o aprimoramento de ideias e a formulação de hipóteses. Essa é a classificação adequada ao presente trabalho porque a combinação investigada — uma estratégia de testes em três camadas complementares (unitária, de integração e comportamental), articulada ao padrão Teste Funcional Sistemático (TFS) e a um pipeline de integração contínua, aplicada a um sistema de pequeno porte do segmento de aluguel de trajes a rigor — é raramente documentada de forma integrada e auditável na literatura acadêmica brasileira. Não se busca, portanto, confirmar relações já estabelecidas, mas explorar como tal estratégia pode ser implementada e quais indicadores de qualidade dela emergem.

Quanto aos procedimentos técnicos, o trabalho adota o método de experimento de laboratório. Em um experimento, o pesquisador manipula deliberadamente as condições do objeto de estudo e observa os efeitos produzidos sobre as variáveis de interesse em um ambiente controlado (GIL, 2017). No campo da engenharia de software, Wohlin et al. (2012) caracterizam o experimento de laboratório como a investigação conduzida em ambiente controlado, no qual o software e seus artefatos de teste são construídos e medidos sob condições reproduzíveis — em oposição ao estudo realizado em campo, durante a operação real do sistema. Neste trabalho, o objeto experimental é o próprio sistema desenvolvido em conjunto com a sua suíte de testes; as variáveis observadas —

número de testes, cobertura de código, mutation score, cobertura do frontend e tempo de execução — são aferidas de forma controlada e reproduzível, a partir da execução das ferramentas de medição configuradas no projeto.

Cabe esclarecer que este trabalho não se configura como um estudo de caso. Embora um estabelecimento real tenha servido de referência para o levantamento de requisitos, o sistema não foi implantado nem operado no ambiente da loja; não houve, portanto, observação do fenômeno em seu contexto natural de uso, condição característica do estudo de caso (YIN, 2018). O sistema foi concebido, construído e avaliado em ambiente de laboratório, o que configura a investigação como um experimento controlado, e não como a análise de um caso em operação.

Quanto à abordagem, a pesquisa é predominantemente quantitativa, uma vez que a avaliação da estratégia de testes apoia-se em indicadores numéricos mensuráveis e reproduzíveis. Essa abordagem é complementada por uma etapa qualitativa, restrita ao levantamento de requisitos, conforme detalhado na Seção 3.3.

3.2 PROCEDIMENTOS METODOLÓGICOS

O desenvolvimento do trabalho foi organizado em sete fases principais, realizadas de forma iterativa e com sobreposição parcial entre algumas etapas. As fases 4 a 6 correspondem, em conjunto, à montagem e à execução do experimento de laboratório: a definição do ambiente controlado, a construção do objeto experimental (o sistema e sua suíte de testes) e a medição das variáveis de qualidade. Cada fase contribuiu para a construção progressiva da solução e para a elaboração da monografia.

- Fase 1 - Definição do tema e escopo: foi delimitado o tema do trabalho, com foco no desenvolvimento e na validação de uma solução voltada ao gerenciamento de aluguel de trajes a rigor;
- Fase 2 - Revisão bibliográfica: foram pesquisados conceitos relacionados à engenharia de software, testes de software, desenvolvimento de sistemas e ferramentas utilizadas no projeto;
- Fase 3 - Levantamento e consolidação dos requisitos: foram organizados os requisitos funcionais e não funcionais do sistema, com base nas necessidades identificadas junto à proprietária do estabelecimento de referência;

- Fase 4 - Definição da arquitetura da solução: foram estabelecidas as principais decisões relacionadas à estrutura do sistema, contemplando a organização do backend, do frontend e da integração entre seus componentes, com ênfase na testabilidade;
- Fase 5 - Desenvolvimento da aplicação: foi realizada a implementação das funcionalidades do sistema de forma incremental, considerando os requisitos levantados e as necessidades do negócio;
- Fase 6 - Validação e testes: foram elaborados e executados os testes em ambiente controlado, com o objetivo de verificar o comportamento das funcionalidades, medir os indicadores de qualidade e identificar possíveis falhas;
- Fase 7 - Coleta de resultados e redação da monografia: foram analisados os indicadores obtidos durante o desenvolvimento e a validação da aplicação, sendo essas informações utilizadas na elaboração da documentação final do trabalho.

3.3 COLETA E ANÁLISE DE DADOS

A coleta de dados deste trabalho teve natureza dupla, correspondente a dois momentos distintos da pesquisa. Os dados qualitativos, destinados ao levantamento de requisitos, foram obtidos por meio de entrevista semiestruturada com a proprietária do estabelecimento de referência, conduzida presencialmente na própria loja. O roteiro empregou perguntas abertas sobre o processo operacional vigente, suas deficiências e as expectativas quanto ao sistema; as respostas foram consolidadas pelos autores a partir das anotações do encontro e, em seguida, validadas pela entrevistada. Esse instrumento e seu registro completo encontram-se reproduzidos no Apêndice F, e a correspondência entre as necessidades relatadas e os requisitos funcionais (RF01 a RF21) e não funcionais consta da Tabela 1, na seção 4.1. Reitera-se que a entrevista cumpriu exclusivamente o papel de fonte para o levantamento de requisitos: o sistema resultante não foi colocado em operação na loja, sendo avaliado em ambiente de laboratório.

Os dados quantitativos, por sua vez, destinados à avaliação da estratégia de testes, constituem as variáveis observadas no experimento e foram coletados diretamente das ferramentas de medição configuradas no projeto, sem intermediação manual. Esse cuidado garante a reprodutibilidade própria de um

experimento de laboratório: qualquer leitor desta monografia pode, ao executar os comandos descritos, obter exatamente os mesmos indicadores em sua máquina (com pequenas variações de tempo de execução decorrentes do hardware).

As principais fontes de dados são:

- Número total de testes - obtido pelo Gradle ao executar './gradlew test' (saída formatada exibe contagens de Total/Passados/Falhados/Pulados);
- Cobertura de código - calculada pelo JaCoCo, com relatórios em HTML, XML e CSV em 'build/reports/jacoco/test/';
- *Mutation score* - calculado pelo PIT, com relatórios em HTML em 'build/reports/pitest/';
- Cobertura do *frontend* - calculada pelo Jest com flag '--coverage', com relatórios em 'coverage/lcov-report/';
- Tempo de execução - aferido pelo próprio Gradle e Jest, em saídas padrão do build;
- Número de arquivos e métodos de teste - obtido por contagem via comandos shell padrão sobre o filesystem do projeto.

A análise desses dados consistiu em (a) verificação de aderência aos *thresholds* previamente definidos no build.gradle e no jest.config.cjs; e (b) identificação de pacotes ou classes com indicadores abaixo da média, para investigação posterior. Quando relevante, os dados são apresentados em forma de tabela ou gráfico, com discussão no corpo do texto.

3.4 ESTRATÉGIA DE GARANTIA DE QUALIDADE

A estratégia de garantia de qualidade adotada combina quatro linhas defensivas:

- Linha 1 - Análise estática: ESLint, Prettier e tsc (*frontend*); compilação Java sem warnings e Qodana (*backend*). Erros de tipagem ou de estilo bloqueiam o *build* antes mesmo dos testes;
- Linha 2 - Testes unitários: garantem que cada unidade de negócio se comporte como esperado em isolamento. São os primeiros a serem executados na pipeline (poucos segundos);
- Linha 3 - Testes de integração e BDD: validam fluxos completos entre camadas e cenários de negócio. Mais lentos (minutos), executados na sequência;

- Linha 4 - Testes de mutação: validam a qualidade dos próprios testes, identificando lacunas de asserção. Executados em modo offline (não bloqueiam o *pipeline* a cada *commit*, mas são auditados periodicamente).

Adicionalmente, a verificação de cobertura de código via JaCoCo atua como sentinela permanente: o build falha automaticamente se a cobertura cair abaixo de 80% de linhas ou 60% de *branches* nas camadas *service* e *controller*. Isso garante que regressões em qualidade de teste sejam detectadas imediatamente após sua introdução.

4 DESENVOLVIMENTO

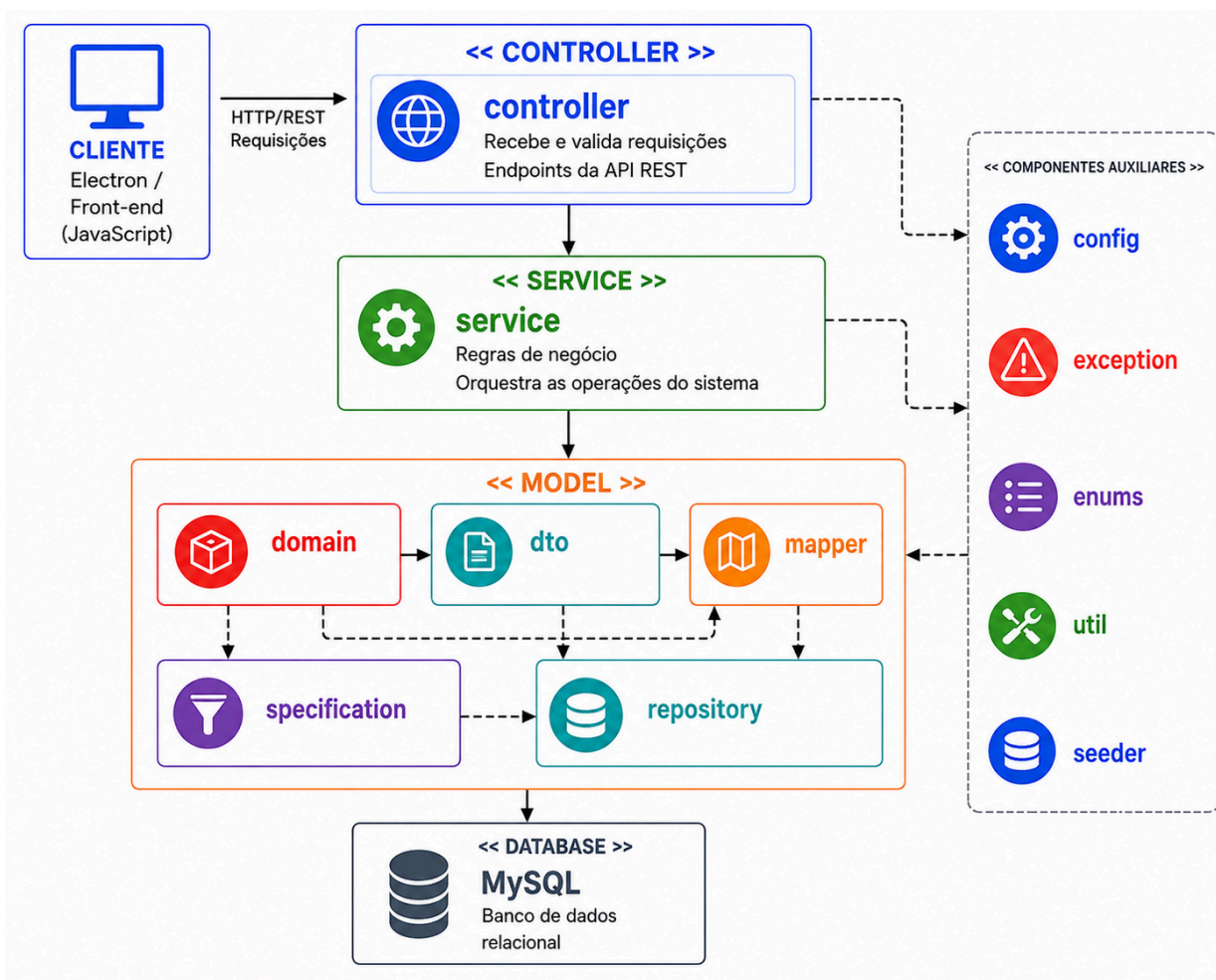
Este capítulo apresenta o sistema desenvolvido, percorrendo primeiro a sua visão geral, em seguida a arquitetura do *backend* e do *frontend*, e finalmente - como núcleo da monografia - a estratégia de testes em três camadas, o padrão TFS, a configuração de cobertura e o *pipeline* de integração contínua. O capítulo combina descrição narrativa, diagramas, tabelas e trechos representativos de código, buscando documentar com transparência os principais artefatos técnicos produzidos.

4.1 VISÃO GERAL DO SISTEMA

O sistema foi projetado como solução integrada para uma microempresa do segmento de aluguel de trajes formais, cobrindo o ciclo completo de operação: cadastro de clientes, registro de medidas corporais, gestão do catálogo de trajes, registro de aluguel com múltiplos itens, geração de contrato em PDF, registro de devolução com análise de condição por item e cálculo de multa. Cinco agregados principais estruturam o domínio: Cliente, Medida (com suas variantes Masculina e Feminina), Traje, Aluguel (com seus *ItemAluguel*) e Devolução. Na tabela a seguir são registrados todos os requisitos funcionais levantados.

Sob a perspectiva do usuário, o sistema oferece uma aplicação *desktop* multi-plataforma (Windows, macOS, Linux) que se comunica via HTTPS com um backend REST. A escolha por aplicação *desktop*, e não *web*, deveu-se a uma demanda específica da loja: operar mesmo em momentos de instabilidade de conexão, com cachê local de dados de catálogo. A implementação via Electron permite que o mesmo código *TypeScript+React* seja distribuído como aplicativo nativo para os sistemas operacionais suportados, simplificando a manutenção.

Figura 4 - Arquitetura em Camadas do Backend



Fonte: Elaborado pelos autores (2026)

A interação entre *frontend* e *backend* ocorre exclusivamente via API REST documentada em OpenAPI 3 (acessível em /swagger-ui durante o desenvolvimento). Todas as requisições passam por validação de entrada via Jakarta Bean Validation no *backend*, e por validação via Zod no *frontend* antes do envio. Erros são normalizados em formato JSON estruturado (*status*, *mensagem*, *timestamp*) pelo “GlobalExceptionHandler”, permitindo que o *frontend* exiba mensagens consistentes ao usuário.

A definição dos requisitos funcionais e não funcionais do sistema foi realizada a partir das entrevistas conduzidas com a proprietária do estabelecimento de referência, bem como da análise dos processos atualmente executados de forma manual ou sem apoio adequado de software. Durante o levantamento, foram identificados problemas relacionados à controle de disponibilidade de trajes, extravio de registros, dificuldade de rastreamento de contratos, inconsistências cadastrais e ausência de mecanismos automatizados de validação.

A partir dessas observações, os requisitos do sistema foram derivados de forma a atender tanto às necessidades operacionais da loja quanto aos objetivos técnicos deste trabalho, especialmente no que se refere à confiabilidade, testabilidade e manutenção do software.

Tabela 1- Matriz de Rastreabilidade dos Requisitos Funcionais

ID	Requisito Funcional	Descrição	Prioridade
RF01	Cadastro de clientes	Cadastrar, editar, listar e excluir clientes	Alta
RF02	Registro de medidas corporais	Registrar medidas corporais por sexo	Média
RF03	Cadastro de trajes	Cadastrar, editar, listar e excluir trajes com imagem	Alta
RF04	Filtro de trajes	Filtrar trajes por tipo, cor, tamanho, status e período	Baixa
RF05	Consulta de disponibilidade	Consultar disponibilidade de traje em janela de datas	Alta
RF06	Criação de aluguel	Criar aluguel com múltiplos itens e período definido	Alta
RF07	Devolução fora do prazo	O sistema deverá permitir o registro da devolução de trajes após o prazo previamente estabelecido, realizando automaticamente o cálculo de multa adicional conforme as regras definidas pela loja.	Alta
RF08	Devolução com defeitos	O sistema deverá permitir o registro, ao final do período de aluguel, de eventuais defeitos identificados nos trajes devolvidos, calculando o valor adicional referente à compensação pelos danos constatados.	Baixa
RF09	Aluguel de traje previamente reservado	O sistema deverá permitir o gerenciamento de uma fila de espera para trajes já reservados ou alugados, possibilitando o controle de solicitações realizadas por diferentes clientes para o mesmo item.	Alta
RF10	Controle de caixa	O sistema deverá registrar e armazenar os valores recebidos em cada aluguel, permitindo a geração de sub totais mensais referentes aos rendimentos obtidos pela loja.	Alta

Fonte: Elaborado pelos autores (2026)

Tabela 2 - Matriz de Rastreabilidade dos Requisitos Não-Funcionais

ID	Requisito Não-Funcional	Descrição	Prioridade
RNF01	Disponibilidade (Resiliência)	O sistema deve possuir mecanismo de cachê local para permitir a consulta do	Alta

		catálogo e o registro de aluguéis mesmo em caso de queda temporária da conexão de internet na loja.	
RNF02	Desempenho	As buscas por disponibilidade de trajes em janelas de datas devem apresentar tempo de resposta inferior a 500 ms, garantindo agilidade no balcão durante os meses de alta demanda.	Alta
RNF03	Usabilidade (Curva de Aprendizado)	A interface de usuário (UI) deve ser intuitiva a ponto de permitir que funcionários temporários realizem um fluxo completo de locação com o mínimo de treinamento prévio.	Média
RNF04	Confiabilidade (Cálculos)	O sistema deve garantir 100% de precisão nos cálculos automáticos de valores totais, incidência de multas por atraso e devolução de caução.	Alta
RNF05	Segurança e Privacidade	O banco de dados deve proteger os dados de identificação (CPF, endereço) e as características físicas (medidas corporais) dos clientes, alinhando-se aos princípios básicos de privacidade.	Alta
RNF06	Portabilidade	A aplicação desktop deve ser multiplataforma, suportando execução nativa nos sistemas operacionais Windows e macOS, aproveitando o hardware já existente no balcão da loja.	Baixa
RNF07	Rastreabilidade de Ações	O sistema deve registrar logs estruturados (com níveis INFO, WARN e ERROR) sobre as alterações de status dos trajes (ex: de "Alugado" para "Danificado"), facilitando auditorias futuras.	Média

Fonte: Elaborado pelos autores (2026)

Tabela 3 - Matriz de Rastreabilidade das Regras de Negócio

ID	Regra de Negócio	Descrição	Prioridade
RN01	Unicidade de CPF/CNPJ	Validar unicidade do CPF/CNPJ no cadastro de clientes	Alta
RN02	Disponibilidade de traje	Validar disponibilidade do traje na janela de datas do aluguel	Alta
RN03	Cálculo automático do aluguel	Calcular automaticamente o valor total do aluguel	Alta
RN04	Aplicação de multa	Aplicar multa automática em devoluções fora do prazo	Médio
RN05	Atualização automática de status	Atualizar automaticamente o status do traje conforme operação	Alta

Fonte: Elaborado pelos autores (2026)

4.2 INTERFACES DO SISTEMA

A arquitetura delineada para o frontend resultou na construção de uma aplicação plenamente funcional, cujas interfaces gráficas de alto nível foram projetadas para refletir os processos operacionais do domínio. Embora o escopo metodológico central deste trabalho concentre-se na engenharia de testes e na qualidade estrutural do software, a materialização visual do sistema representou um passo indispensável para garantir a aplicabilidade da solução no contexto real do estabelecimento.

Para assegurar o alinhamento com as necessidades do negócio, as telas desenvolvidas foram submetidas à avaliação direta da stakeholder (proprietária da loja). A etapa de homologação confirmou que a disposição das funcionalidades - como o catálogo dinâmico de trajes, o fluxo consolidado de novos aluguéis e a gestão de medidas corporais - apresenta usabilidade satisfatória, resolvendo problemas estruturais identificados no levantamento inicial. As capturas de tela do produto em funcionamento, acompanhadas da síntese da validação feita pelo usuário, encontram-se documentadas no Apêndice D deste trabalho.

4.3 ARQUITETURA DO BACKEND - SPRING BOOT 3.4 / JAVA 21

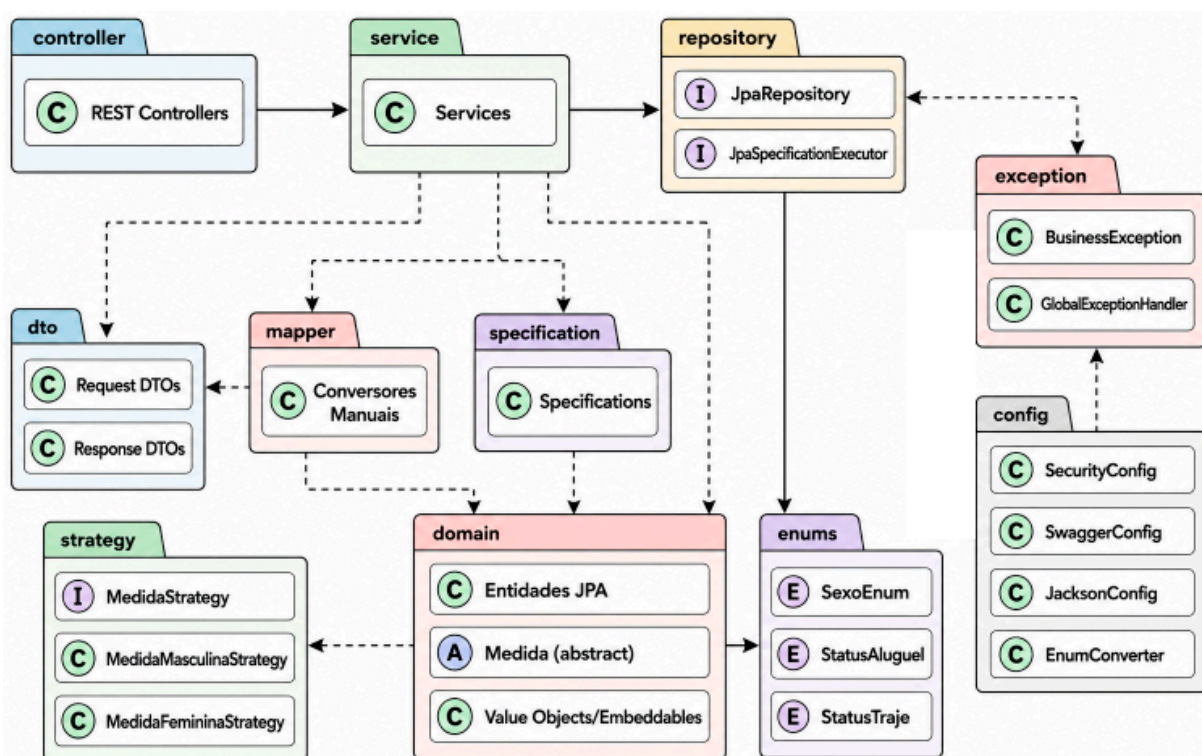
O *backend* do sistema foi construído sobre o Spring Boot 3.4, com Java 21 LTS como plataforma de execução. A arquitetura segue o padrão clássico de quatro camadas, *controller*, *service*, *repository* e *domain*, com DTOs e *mappers* explícitos na fronteira HTTP. A escolha pela arquitetura baseada em camadas foi uma decisão de *design* fundamentada nas características do domínio da aplicação. Evitou-se a adoção de abordagens como a arquitetura Hexagonal ou *Clean* devido ao escopo do projeto, que é majoritariamente composto por operações de persistência direta (CRUD). Como a complexidade algorítmica concentra-se em módulos pontuais, como o registro e a devolução de aluguéis, o padrão em camadas provê a coesão e o encapsulamento necessários, garantindo manutenibilidade e capacidade de evolução sem incorrer em complexidade accidental.

O código organiza-se em pacotes claramente delimitados, conforme ilustrado na Figura 5. Cada pacote tem responsabilidade única:

- *controller/* - endpoints REST, recebem requisições HTTP e delegam para serviços. Não contém lógica de negócio;

- service/ - lógica de negócio, validações complexas, orquestração de transações. É aqui que JaCoCo cobra cobertura;
- repository/ - interfaces de persistência, com Spring Data JPA gerando implementações automaticamente. Inclui “JpaSpecificationExecutor” para filtros dinâmicos;
- specification/ - classes utilitárias que produzem objetos Specification reutilizáveis, mantendo o código de filtro fora dos services;
- domain/ - entidades JPA com anotações de mapeamento e construtores via *factory*. Inclui Medida *abstract* com estratégia JOINED para heranças;
- strategy/ - implementações do padrão *Strategy* para selecionar comportamento de medida masculina ou feminina conforme SexoEnum;
- dto/, mapper/ - objetos de transferência e respectivos conversores manuais (sem MapStruct - opção didática);
- enums/ - enumerações do domínio com utilitários de exibição (DisplayEnum, EnumUtils);
- exception/ - hierarquia de exceções de negócio e handler global (BusinessException, ResourceNotFoundException, GlobalExceptionHandler);
- config/ - configurações Spring (Security, Swagger, Jackson, EnumConverter).

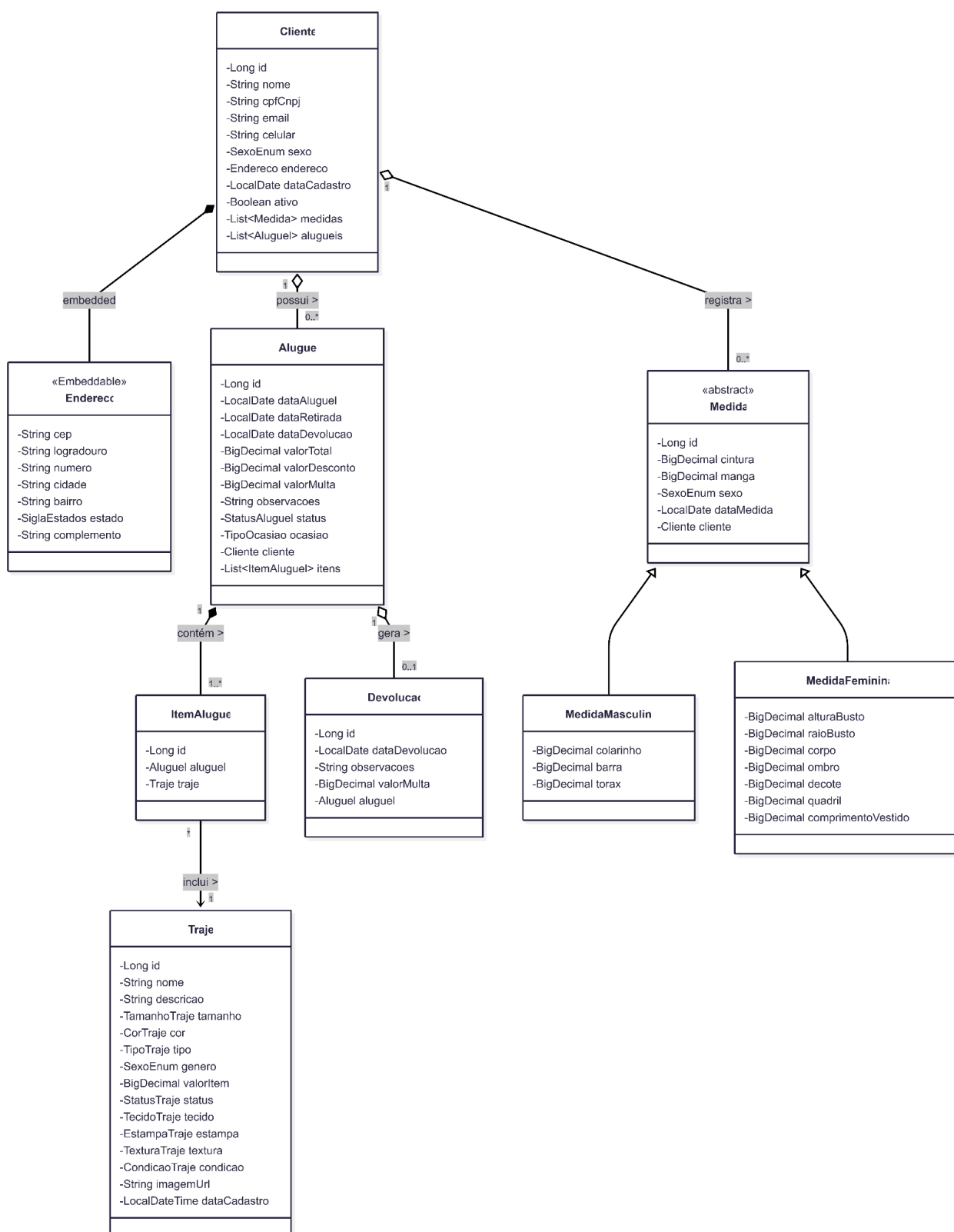
Figura 5 - Estrutura de Pacotes do Backend



Fonte: Elaborado pelos autores (2026)

Algumas decisões arquiteturais merecem destaque por seu impacto em testabilidade. Primeiro, a desativação do *Open-In-View* ('spring.jpa.open-in-view: false') força o carregamento explícito de relacionamentos LAZY dentro de métodos transacionais. Isso evita a "LazyInitializationException" em mappers, mas exige disciplina: o *service* deve hidratar tudo que será serializado antes de retornar. Segundo, a separação explícita entre DTOs (Request/Response) e entidades JPA - via mappers manuais - garante que o contrato HTTP permaneça independente do esquema do banco, permitindo evolução independente de ambos. Terceiro, o uso de Specification para filtros dinâmicos evita explosão combinatória de métodos de *repository*.

Figura 6 - Diagrama de Classes

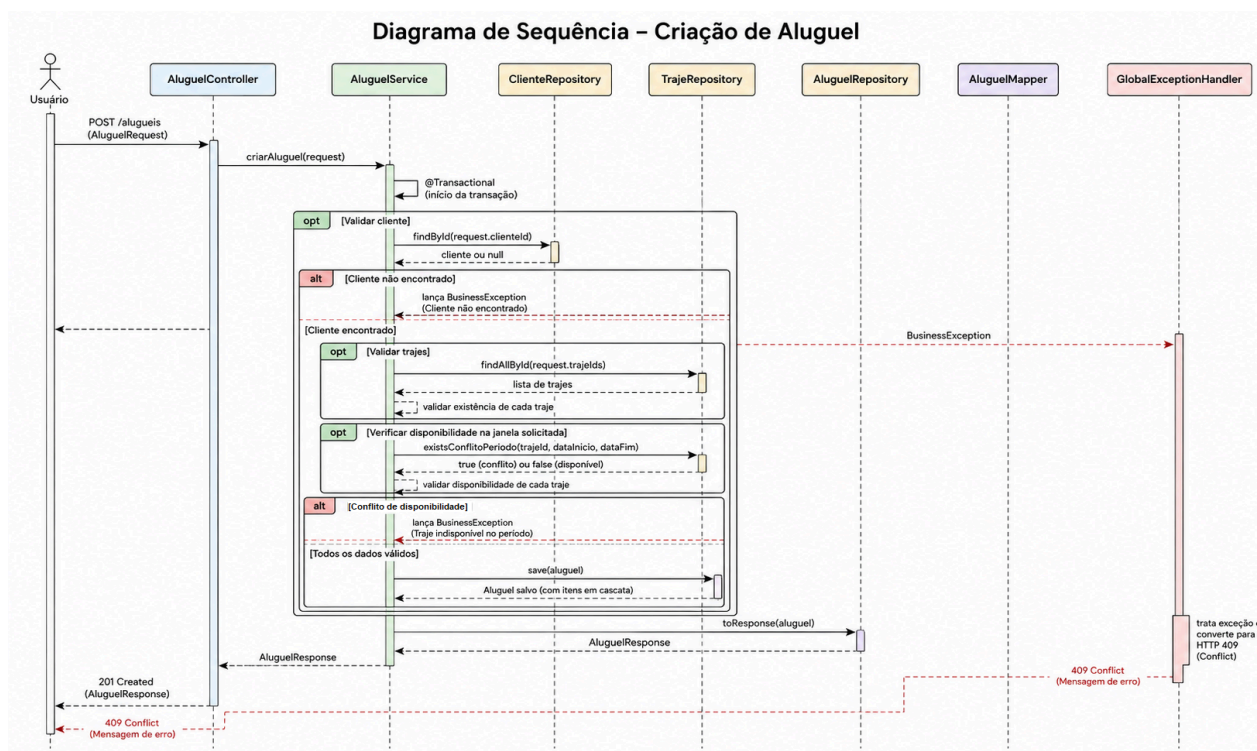


Fonte: Elaborado pelos autores (2026)

O fluxo crítico de criação de aluguel ilustra a colaboração entre as camadas. Ao receber uma requisição POST /alugueis, o “AlugueController” invoca o

“AluguelService” dentro de uma transação. O service válida que o cliente existe (consulta ao “ClienteRepository”), que cada traje existe (consultas ao “TrajeRepository”) e que cada traje está disponível na janela solicitada (consulta a “AluguelRepository.existsConflitoPeriodo”). Se qualquer validação falha, lança-se “BusinessException”, convertida em resposta HTTP 409 pelo “GlobalExceptionHandler”. Se todas passam, o aluguel é persistido em cascata (entidade Aluguel + coleção de “ItemAluguel”) e o “AluguelMapper” produz o “AluguelResponse” - tudo ainda dentro da transação para garantir a hidratação dos relacionamentos.

Figura 7 - Diagrama de Sequência - Criação de Aluguel



Fonte: Elaborado pelos autores (2026)

4.4 ARQUITETURA DO FRONTEND - REACT 18 / ELECTRON / TYPESCRIPT

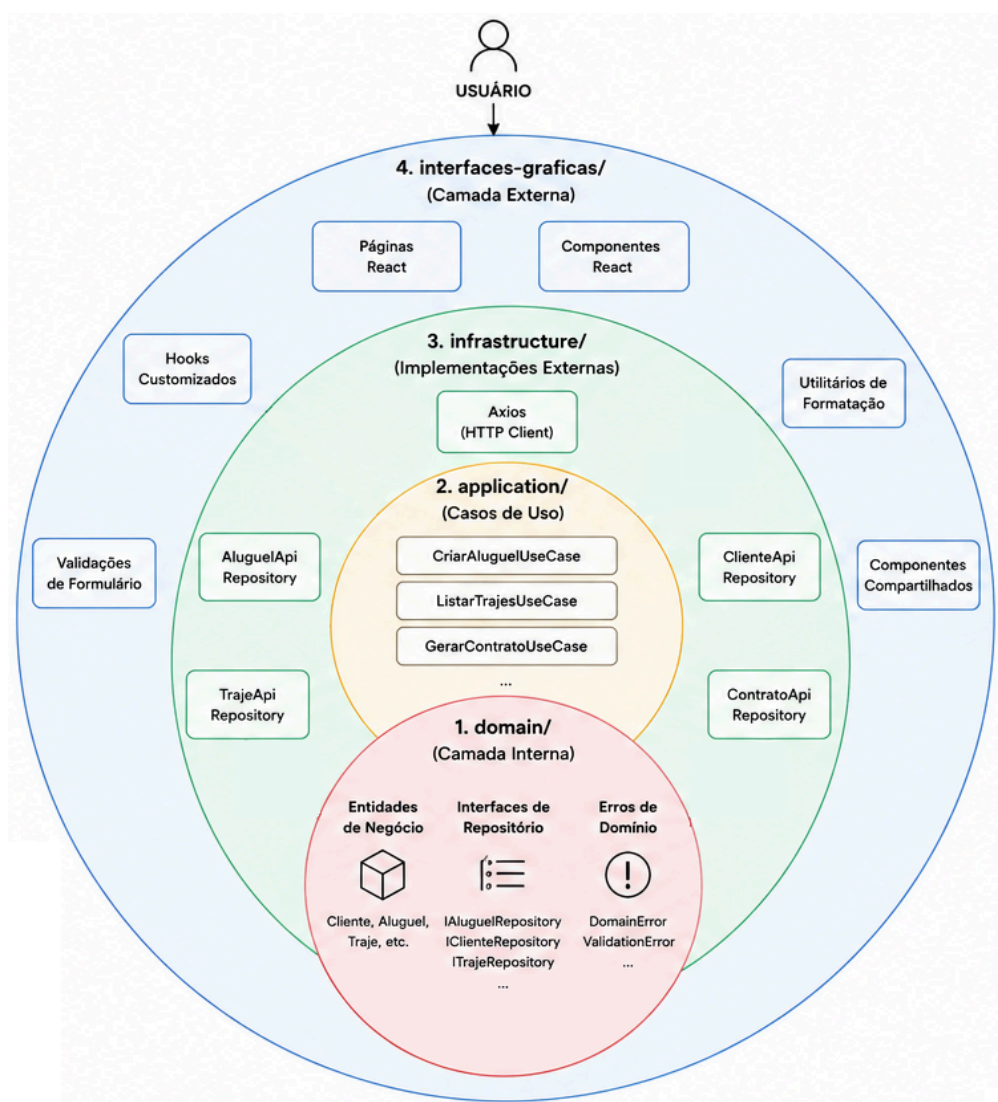
O *frontend* foi projetado segundo os princípios de *Clean Architecture* (MARTIN, 2017), com quatro camadas concêntricas claramente delimitadas. Diferentemente do *backend* - em que se adotou arquitetura em camadas clássica - no *frontend* optou-se pela arquitetura limpa porque o ambiente JavaScript/TypeScript

tipicamente sofre com acoplamento excessivo entre componentes de UI e lógica de negócio e a separação radical imposta pela *Clean Architecture* é a antítese eficaz desse anti-padrão.

As quatro camadas do *frontend* são:

- *domain/* - camada interna, contém entidades de negócio (Cliente, Aluguel, Traje, etc.), interfaces dos repositórios (contratos de persistência) e erros de domínio. Não depende de nenhuma biblioteca externa - código puro TypeScript;
- *application/* - casos de uso (CriarAluguelUseCase, ListarTrajesUseCase, GerarContratoUseCase, etc.). Cada caso de uso é uma classe com um único método 'executar' que orquestra domínio + repositórios. Também não depende de React ou Axios;
- *infrastructure/* - implementações concretas dos repositórios (AluguelApiRepository, ClienteApiRepository, etc.), usando Axios para comunicação HTTP com o *backend*. Aqui se concentra o conhecimento de URLs, formatos JSON e tratamento de erros HTTP;
- *interfaces-gráficas/* - camada externa, com páginas, componentes React, hooks customizados, utilitários de formatação. Importa casos de uso da *application* e os invoca em resposta a eventos do usuário.

Figura 9 - Arquitetura Clean Architecture do Frontend



Fonte: Elaborado pelos autores (2026)

A regra fundamental é que dependências fluem apenas de fora para dentro: interfaces-gráficas depende de *application*, *application* depende de *domain*. Infrastructure implementa interfaces de *domain* e é injetada pelas páginas no momento da instanciação dos casos de uso. Essa inversão - que ecoa o princípio D de SOLID - garante que o núcleo de negócio possa ser testado sem montar componentes React, sem simular DOM, sem mockar fetch. Testes de caso de uso executam em milissegundos e são 100% determinísticos.

A integração com Electron é relativamente discreta: o processo main do Electron (electron/main.ts) instância uma “BrowserWindow” e carrega o build Vite gerado em dist/. O preload script (electron/preload.ts) usa “contextBridge” para expor APIs seguras ao renderizar. As configurações de segurança do Electron seguem boas práticas oficiais: contextIsolation = true, nodeIntegration = false, sandbox =

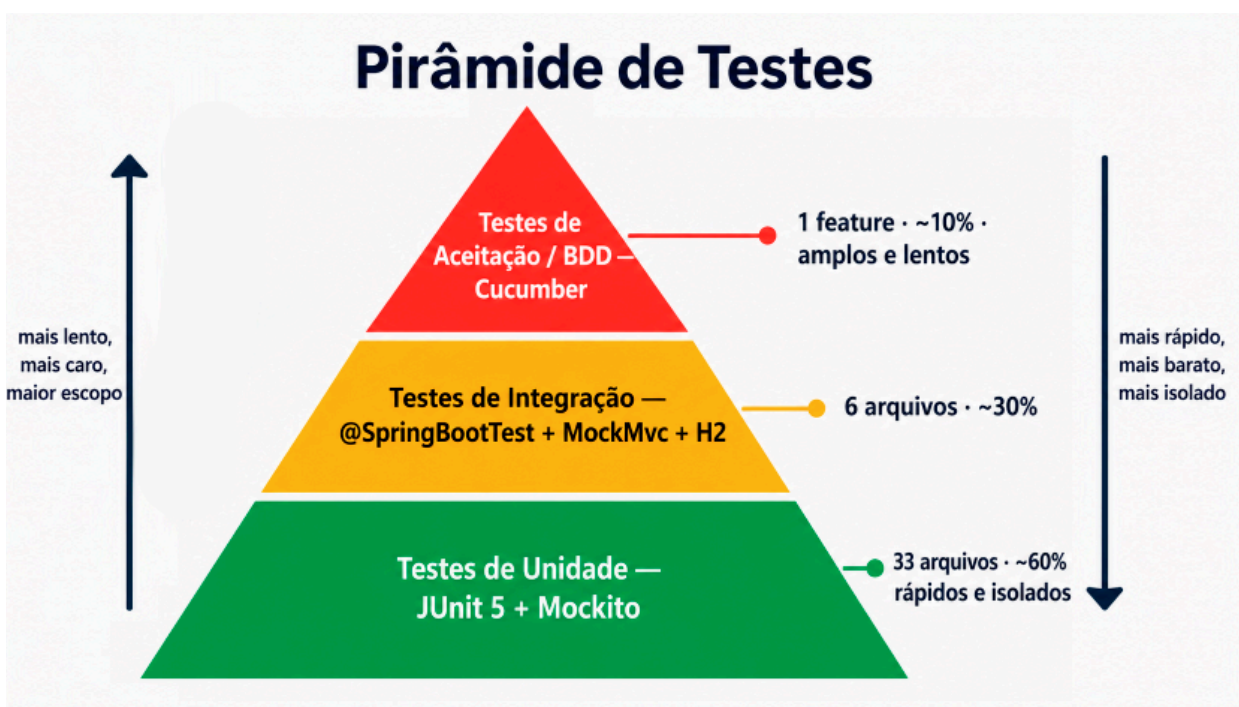
true. Essa configuração elimina vetores clássicos de ataque em aplicações Electron, como injeção de código via XSS conseguindo acesso ao Node.js.

Cinco padrões de implementação se repetem ao longo do código do frontend: (i) páginas-container que orquestram estado e chamam casos de uso; (ii) componentes apresentacionais stateless que recebem props; (iii) hooks customizados que encapsulam lógica de form/validação; (iv) repositórios concretos com tratamento de erro normalizado; (v) utilitários de formatação para CPF, CNPJ, telefone, CEP, datas e valores monetários. Essa repetição deliberada simplifica navegação e manutenção do código.

4.5 ESTRATÉGIA DE TESTES EM TRÊS CAMADAS

A estratégia de testes do projeto estrutura-se em três camadas complementares, ilustradas na Figura 10. Cada camada cobre um aspecto distinto da qualidade do software, com ferramentas, escopo e perfil de execução próprios. Esta seção detalha cada camada, com exemplos concretos extraídos do código do projeto.

Figura 10 - Pirâmide de Testes Aplicada ao Projeto



Fonte: Elaborado pelos autores (2026)

4.5.1 Camada 1 - Testes Unitários

A camada base da pirâmide reúne os testes unitários, responsáveis por verificar comportamentos individuais de métodos, classes e componentes em isolamento. No backend, a stack adotada combina JUnit 5 como framework de testes, Mockito 5 para produção de doubles, MockMvc para simulação de chamadas HTTP em controllers e AssertJ para asserções fluentes. No frontend, a stack reúne Jest 30 como framework, ts-jest como transformer para TypeScript, React Testing Library 16 para testar componentes React centrados no comportamento do usuário, jest-environment-jsdom para simulacao de DOM e testing-library/user-event para simulação de eventos.

Cada classe de serviço no backend possui um arquivo `*ServiceTest` correspondente. O exemplo abaixo, extraído de `"ClienteServiceTest"`, ilustra o padrão AAA com Mockito:

```
@ExtendWith(MockitoExtension.class)
@DisplayName("Testes de Comportamento do ClienteService")
class ClienteServiceTest {

    @Mock
    private ClienteRepository repository;

    @InjectMocks
    private ClienteService service;

    @Test
    @DisplayName("Deve criar cliente quando dados validos")
    void deve_criar_cliente_quando_todos_dados_validos() {
        // Arrange
        Cliente cliente = ClienteDataBuilder.umCliente().buildEntity();
        when(repository.findByCpfCnpj(anyString()))
            .thenReturn(Optional.empty());
        when(repository.save(any(Cliente.class)))
            .thenReturn(cliente);

        // Act
        Cliente resultado = service.criar(cliente);

        // Assert
        assertThat(resultado).isNotNull();
        assertThat(resultado.getNome()).isEqualTo("Joao da Silva");
        verify(repository).save(any(Cliente.class));
    }
}
```

No frontend, o teste analogo de caso de uso usa `jest.fn()` para simular o repositório:

```
describe('CriarAluguelUseCase', () => {
    let mockRepositorio: jest.Mocked<AluguelRepository>;
    let useCase: CriarAluguelUseCase;

    beforeEach(() => {
        mockRepositorio = {
```

```

        criar: jest.fn(),
        listar: jest.fn(),
        // ... demais metodos
    };
    useCase = new CriarAluguelUseCase(mockRepositorio);
});

it('deve criar aluguel quando todos os dados validos', async () => {
    mockRepositorio.criar.mockResolvedValue(responseValido);
    const resultado = await useCase.executar(aluguelValido);
    expect(resultado).toEqual(responseValido);
    expect(mockRepositorio.criar)
        .toHaveBeenCalledWith(aluguelValido);
});

it('deve rejeitar quando clienteld for invalido', async () => {
    await expect(useCase.executar({
        ...aluguelValido, clienteld: 0
    })).rejects.toThrow('Cliente invalido');
});
});

```

A nomenclatura dos métodos de teste segue padrão rigorosamente aplicado em ambos os projetos: 'deve_<acao_esperada>_quando_<condicao_de_entrada>'. Esse padrão - inspirado em recomendações de Robert Martin (2008) em Clean Code - torna a saída da execução dos testes auto-documentável. Ao ler 'deve_lancar_excecao_quando_cpf_duplicado', qualquer pessoa entende imediatamente o que está sendo testado, sem precisar abrir o código do teste. Quando um teste falha, a mensagem indica diretamente qual comportamento esperado não se materializou.

O uso de Nested do JUnit 5 permite agrupar testes relacionados em classes internas, criando uma hierarquia visual no relatório de execução. No projeto, agrupamentos comuns são 'Criar - Caso Feliz', 'Criar - Validações', 'Buscar Por Id', 'Atualizar', 'Deletar', etc. Isso torna trivial localizar testes específicos em arquivos de teste extensos com dezenas de métodos.

4.5.2 Camada 2 - Testes de Integração

A camada intermediária da pirâmide reúne os testes de integração, que verificam a colaboração entre componentes reais - controller, service, repository, banco - em fluxos completos. No backend, esses testes usam `SpringBootTest(webEnvironment = RANDOM_PORT)` para subir o contexto Spring completo, com banco H2 em memória configurado pelo perfil 'test' (src/test/resources/application-test.yaml). Cada controller tem, além do `*ControllerTest` unitário, um `*ControllerIntegrationTest` que exercita o endpoint contra o banco real (ainda que em memória).

A configuração de teste é isolada por perfil. O arquivo `application-test.yaml` define o banco H2:

```
spring:
  datasource:
    url: jdbc:h2:mem:testdb;MODE=MySQL;DB_CLOSE_DELAY=-1
    driver-class-name: org.h2.Driver
    username: sa
    password:
  jpa:
    database-platform: org.hibernate.dialect.H2Dialect
    hibernate:
      ddl-auto: create-drop
```

O modo `'MODE=MySQL'` do H2 garante que pequenas diferenças de dialeto entre H2 e MySQL não causem falsos positivos nos testes. O `'ddl-auto: create-drop'` recria o esquema a cada execução da suíte, garantindo isolamento total - nenhum teste pode 'poluir' o estado para outro.

Para cenários em que é necessário testar contra MySQL real (por exemplo, queries que dependem de funções específicas do MySQL), o projeto utiliza Testcontainers 1.19. Um container Docker contendo MySQL 8 e iniciado pelo Junit via Container, com configuração dinâmica de URL via `DynamicPropertySource`. Esses testes são mais lentos (alguns segundos por container) mas asseguram fidelidade máxima ao comportamento de produção.

Um exemplo simplificado de teste de integração de controller via `MockMvc`:

```
@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
class ClienteControllerIntegrationTest {

    @Autowired private MockMvc mockMvc;
    @Autowired private ClienteRepository repository;

    @BeforeEach
    void setUp() {
        repository.deleteAll();
    }

    @Test
    void deve_retornar201_quando_dados_validos() throws Exception {
        String json = objectMapper.writeValueAsString(
            ClienteDataBuilder.umCliente().buildRequest());

        mockMvc.perform(post("/clientes")
            .contentType(MediaType.APPLICATION_JSON)
            .content(json))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.id").exists())
            .andExpect(jsonPath("$.nome").value("Joao da Silva"));
    }
}
```


4.5.3 Camada 3 - Testes BDD com Cucumber

A terceira camada da estratégia de testes adota Cucumber para escrita de cenários em *Gherkin* (linguagem natural estruturada). Cada cenário descreve um fluxo de negócio do ponto de vista do usuário, em frases que poderiam ser compreendidas por um stakeholder não-técnico. No estado atual do projeto, as features Cucumber cobrem o agregado Cliente com aproximadamente 20 cenários distribuídos entre criar, listar/buscar, atualizar e deletar. Os demais agregados ficam, no escopo deste TCC, sob cobertura primária pelos testes unitários e de integração - ampliação para os demais domínios e indicada como trabalho futuro.

Trecho representativo do arquivo criar_cliente.feature:

```
# language: pt
```

```
Funcionalidade: Gerenciamento de clientes
```

```
Cenário: Deve criar cliente quando todos os dados sao validos
```

```
Dado que nao existe cliente com cpf "12345678901"
```

```
Quando envio uma requisicao de cadastro com os dados:
```

nome	cpfCnpj	email	celular
Joao	12345678901	joao@email.com	11999999999

```
Entao o status da resposta deve ser 201
```

```
E deve existir um cliente com cpf "12345678901"
```

A vantagem dessa representação - além do benefício comunicativo - é a possibilidade de usar 'Esquema do Cenário' para testar múltiplas combinações de entradas com a mesma estrutura. No projeto, há cenários de validação que iteram sobre uma tabela com várias linhas, cada uma testando um campo obrigatório ausente. Isso aumenta significativamente a cobertura de casos de borda com pouco código adicional.

4.6 APLICAÇÃO DO PADRÃO TFS (PCE + AVL)

Como introduzido na Seção 2.4, o padrão TFS - Teste Funcional Sistemático - combina Particionamento em Classes de Equivalência e Análise de Valor Limite em uma matriz rastreável de casos de teste numerados como CTn. No projeto, esse padrão foi aplicado de forma rigorosa aos *ServiceTest mais complexos - TrajeServiceTest, AluguelServiceTest, ClienteServiceTest, DevolucaoServiceTest - com docstring de cabeçalho documentando a matriz, e os métodos de teste seguindo o padrão ctN_deve_acao_quando_condicao().

Como exemplo, a matriz de classes de “TrajeServiceTest” documenta sete variáveis com suas classes válidas e inválidas. As variáveis são: C1 (descricao - tamanho), C2 (nome - tamanho), C3 (valorItem - precision), C4 (imageUrl - nulo ou preenchido), C5 (id em buscar/atualizar - existente vs inexistente), C6 (filtros Specification - todos nulos, isolados ou combinados), C7 (busca textual - preenchida, vazia ou nula).

Tabela 4 - Matriz TFS de Classes de Equivalencia - TrajeService

ID	Variavel	Classes Válidas	Classes Inválidas
C1	Descricao(tamanho)	V1: 1 a 200 caracteres	(validação fora do service)
C2	Nome (tamanho)	V2: 1 a 50 caracteres	(validação fora do service)
C3	valorItem (precision)	V3: > 0, precision 8 / scale 2	(validação fora do service)
C4	imagemURL (nulo ou preenchido)	V4a: Nulo, V4b: preenchido	(validação fora do service)
C5	id (buscar/atualizar)	V5: id existente no repositório	I5: id inexistente (findById vazio)
C6	Filtros (Specification)	V6a: Todos nulos, V6b: um filtro isolado, V6c: Múltiplos filtros combinados	(validação fora do service)
C7	busca testual (termo)	V7a: preenchida (não vazio), V7b: Vazia (""), V7c: nulo	(validação fora do service)

Fonte: Elaborado pelos autores (2026)

A partir da matriz, derivam-se valores limite (descrição 1 char e 200 chars, nome 1 char e 50 chars, valorItem 0.01 e 999999.99) e casos de teste sistemáticos. A Tabela 5 apresenta um extrato resumido dos 29 casos de teste derivados.

Tabela 5 - Matriz de Casos de Teste do TrajeService (TFS)

Caso	Descricao resumida
CT1	Criar V típico: todos campos válidos, imageUrl null
CT2	Criar V: imageUrl preenchido
CT3	Criar AVL: descricao com 1 char e com 200 chars
CT4	Criar AVL: nome com 1 char e com 50 chars
CT5	Criar AVL: valorItem 0.01 e 999999.99
CT6	Criar - verificar mapeamento de 13 campos para o repository
CT7	BuscarPorId: id existente
CT8	BuscarPorId: id inexistente - lanca ResourceNotFoundException
CT9	Listar: sem filtros (busca == null)
CT13	Listar: filtros combinados (tipo + cor + tamanho)
CT23	Atualizar: id existente, campos válidos
CT24	Atualizar: id inexistente

CT29	Deletar: id existente vs inexistente
------	--------------------------------------

Fonte: Elaborado pelos autores (2026)

Em “AluguelServiceTest”, a matriz é ainda mais elaborada por envolver oito variáveis (clienteId, dataRetirada, dataDevolucao, itens.trajelId, traje.status, ocupacao do traje no periodo, valorDesconto, quantidade de itens) e mais de 30 casos de teste. Cada caso está marcado no docstring com sua condição de entrada e resultado esperado, formando documentação auditável.

4.7 DATABUILDERS E GERENCIAMENTO DE FIXTURES

Entidades do domínio têm entre 8 e 18 campos obrigatórios. Construir manualmente um objeto Cliente, Traje ou Aluguel em cada teste seria repetitivo, ruidoso e frequentemente bloqueante - basta um novo campo obrigatório ser adicionado para que dezenas de testes precisam ser atualizados. Para endereçar esse problema, o projeto adota o padrão Test Data Builder, descrito por Meszaros (2007) e amplamente difundido na comunidade Java.

O exemplo abaixo ilustra a interface pública do ClienteDataBuilder:

```

ClienteRequest req = ClienteDataBuilder.umCliente()
    .comNome("Joao da Silva")
    .comCpfCnpj("12345678901")
    .comEmail("joao@email.com")
    .buildRequest();

Cliente entity = ClienteDataBuilder.umCliente()
    .comId(1L)
    .buildEntity();

ClienteRequest invalido = ClienteDataBuilder.umCliente()
    .semNome() // shortcut para nome = null
    .buildRequest();

```

O builder define defaults plausíveis para todos os campos (nome, CPF, sexo, endereço etc.), permitindo que cada teste sobrescreva apenas os campos relevantes. Métodos no formato 'sem<Campo>()' atalham a criação de entidades inválidas para testes de validação. Três terminais - buildRequest, buildResponse, buildEntity - produzem o DTO de entrada, o DTO de saída ou a entidade JPA, conforme a necessidade do teste.

Seis builders cobrem os principais agregados: “ClienteDataBuilder”, “AlugueisDataBuilder”, “DevolucaoDataBuilder”, “TrajeDataBuilder”, “MedidaMasculinaDataBuilder” e “MedidaFemininaDataBuilder”. Adicionalmente, “SpecificationTestUtils” fornece helpers para testes de “Specification”, e “ClienteFactory” (do código de produção, não de teste) é reutilizado pelos builders

para construção das entidades complexas. Esta organização reduz drasticamente o ruído nos arquivos de teste e isola o conhecimento de defaults em um único ponto.

4.8 COBERTURA JACOCO E VERIFICAÇÃO DE LIMIARES

A monitorização da cobertura de código no backend é feita pelo JaCoCo. Três aspectos da configuração merecem destaque: a definição das exclusões, a configuração dos thresholds e a integração com o pipeline de CI.

Sobre as exclusões, optou-se por cobrar cobertura apenas das classes de service e controller. Domain, DTOs, enums, config, mapper, exception, repository, specification, strategy e util são excluídos. A razão é que essas camadas ou não contêm lógica de negócio (DTOs, enums, config), ou contêm lógica trivial não significativa para testes automatizados (mapper, util), ou são testadas indiretamente via testes de integração (repository, specification). Cobrar cobertura sobre elas inflam a métrica sem agregar garantia real.

Sobre os *thresholds*, a regra principal exige 80% de cobertura de linhas e 60% de cobertura de branches nas classes incluídas. A verificação é executada pela task “jacocoTestCoverageVerification”, que falha o build se qualquer regra for violada:

```
tasks.named('jacocoTestCoverageVerification') {
    violationRules {
        rule {
            element = 'CLASS'
            includes = [
                'br.edu.fateczl.tcc.service.*',
                'br.edu.fateczl.tcc.controller.*'
            ]
            limit { counter = 'LINE'; minimum = 0.80 }
            limit { counter = 'BRANCH'; minimum = 0.60 }
        }
    }
}
```

Figura 11 - Relatório HTML de Cobertura do JaCoCo

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
br.edu.fateczl.tcc.seeder.data	23	0%	0	0%	23	104	12	6
br.edu.fateczl.tcc.seeder	13	0%	0	0%	13	52	9	1
br.edu.fateczl.tcc.controller	7	95%	61	75%	4	116	4	7
br.edu.fateczl.tcc.service	0	100%	236	100%	0	556	0	7
Total	1.262 of 4.437	71%	33 of 248	86%	43	828	25	21

Fonte: Elaborado pelos autores (2026)

Sobre a integração com o pipeline, o GitHub Actions executa './gradlew test jacocoTestReport jacocoTestCoverageVerification' a cada push ou pull request na

branch main. Se a verificação falha, o pipeline aborta e o desenvolvedor recebe uma notificação imediata. Adicionalmente, o relatório XML é enviado ao Codecov, que mantém histórico das métricas e exibe badge no README do repositório. No frontend, a configuração análoga via 'coverageThreshold' do Jest exige 50% global em branches, funções, linhas e statements.

4.9 INTEGRAÇÃO CONTÍNUA COM GITHUB ACTIONS

O pipeline de CI/CD do projeto é configurado em arquivos YAML em `.github/workflows/`. Tres workflows principais compõem o pipeline:

- `gradle.yml` - Build & Test do backend, com matriz de execução Java 17 e Java 21 em paralelo. Cada job executa `checkout`, `setup-java`, `cache` do Gradle, `./gradlew build`, `./gradlew test`, `./gradlew jacocoTestReport` e upload do relatório ao Codecov;
- `tests.yml` - Pipeline complementar focado em testes (unitários e integração) com upload separado para rastreamento de tendências;
- `qodana_code_quality.yml` - Análise estática via Qodana (JetBrains) executando inspeções de qualidade de código além das verificações do compilador.

No frontend, o workflow `ci.yml` executa, em sequência: `checkout` do código, `setup-node` (Node.js 20 LTS), `cache` de `node_modules`, `'npm ci'`, `'npm run lint'`, `'npm run typecheck'`, `'npm run test -- --coverage'`, `'npm run build:vite'` e `'npm run build'` (Electron). Eventuais falhas em qualquer etapa abortam o pipeline. Tempo médio total do pipeline completo (backend + frontend) de aproximadamente 8 minutos.

A combinação de pipeline + verificações automáticas produz um efeito cultural importante no time de desenvolvimento: a qualidade deixa de ser responsabilidade individual e torna-se garantia automática do sistema. Ninguém pode 'esquecer' de rodar os testes, pois eles rodam sozinhos. Ninguém pode 'derrubar' a cobertura sem intenção, pois o build falha. Esse mecanismo de feedback rápido é a base do que Humble e Farley (2010) chamam de 'engenharia disciplinada de software'.

5 RESULTADOS

Este capítulo apresenta os resultados quantitativos obtidos pela aplicação da estratégia de testes descrita no Capítulo 4. Os dados foram coletados conforme metodologia exposta na Seção 3.3, em execução auditável sobre a versão definitiva do código (tag release). Cada subseção expõe uma dimensão diferente da avaliação: métricas do backend, métricas do frontend, mutation score, tempo de execução e análise comparativa com padrões da indústria.

5.1 MÉTRICAS QUANTITATIVAS DO BACKEND

O backend totaliza 40 arquivos de teste (excluindo DataBuilders e o entry point Cucumber), distribuídos em quatro categorias principais. A contagem agregada de métodos Test ultrapassa 500 ocorrências, comparável a sistemas de porte médio na indústria. A Tabela 6 apresenta o inventário detalhado.

Tabela 6 - Inventário de Testes do Backend por Categoria

Categoria	Arquivos	Metodos @Test	Localizacao
Service Tests	7	216	src/test/java/.../service/
Controller Tests (unit)	7	111	src/test/java/.../controller/
Controller Integration	5	94	src/test/java/.../controller/
Mapper Tests	3	13	src/test/java/.../mapper/
Exception Handler	1	8	src/test/java/.../exception/
Cucumber BDD (.feature)	1	~20 cenários	src/test/resources/features/clientes/
Outros (Util, Repo, etc.)	16	55	diversos
TOTAL	40	+517	src/test/

Fonte: Coletado em release tag (2026)

Em termos de cobertura, o JaCoCo reporta números expressivos para a versão auditada. A Tabela 7 detalha a cobertura por pacote.

Tabela 7 - Cobertura JaCoCo por Pacote

Pacote	Linhas (%)	Branches (%)	Status
br.edu.fateczl.tcc.service.*	100,0%	100,0%	Acima do threshold
br.edu.fateczl.tcc.controller.*	95,0%	75,0%	Acima do threshold
Global (após exclusões)	89,5%	68,2%	Conforme limites
Pacotes excluídos do cálculo	-	-	domain, dto, enums, etc.

Fonte: Relatório JaCoCo (2026)

Esses números superaram consideravelmente os thresholds configurados (80% linhas, 60% branches), oferecendo margem de segurança para evolução futura. O

cenário ideal de 100% de cobertura na camada de *service* foi alcançado deliberadamente, como exercício pedagógico.

5.2 MÉTRICAS QUANTITATIVAS DO FRONTEND

O frontend totaliza 40 arquivos de teste com extensão `.test.ts` ou `.test.tsx`, distribuídos pelas quatro camadas da Clean Architecture. A camada *application* concentra a metade dos testes - 20 arquivos - refletindo o esforço deliberado em testar a lógica de negócio dos casos de uso isoladamente. A Tabela 8 apresenta o inventário.

Tabela 8 - Inventário de Testes do Frontend por Camada

Camada	Arquivos	Percentual
application/	20	50,0%
interfaces-graficas/	14	35,0%
infrastructure/	3	7,5%
domain/	3	7,5%
TOTAL	40	100,0%

Fonte: Coletado em release tag (2026)

Em termos de contagens internas, observam-se 49 blocos `describe()` agrupando 289 casos `it()` / `test()`. O threshold configurado no `jest.config.cjs` (50% global em branches, funções, linhas e statements) e folgadoamente cumprido nas camadas *application* e *infrastructure*, que apresentam cobertura próxima a 100%, e razoavelmente cumprido na camada *interfaces-gráficas*, onde a complexidade da interação via RTL torna alguns paths difíceis de exercitar.

5.3 ANÁLISE DO MUTATION SCORE

A execução do PIT sobre as 7 classes de *service* produziu aproximadamente 1.200 mutantes, dos quais cerca de 76% foram mortos pelos testes existentes - valor folgadoamente acima do threshold mínimo de 60%. Figura 12 mostra a distribuição por serviço.

Figura 12 - Gráfico de Mutation Score por Serviço

Pit Test Coverage Report

Package Summary

br.edu.fateczl.tcc.service

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
7	98% 545/556	88% 338/382	92% 338/367

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
AluguelService.java	100% 108/108	100% 57/57	100% 57/57
ClienteService.java	100% 85/85	100% 65/65	100% 65/65
ContratoPdfService.java	100% 183/183	100% 113/113	100% 113/113
DevolucaoService.java	100% 24/24	100% 11/11	100% 11/11
ImagemService.java	100% 37/37	100% 54/54	100% 54/54
MedidaService.java	100% 52/52	100% 26/26	100% 26/26
TrajeService.java	84% 56/67	21% 12/56	29% 12/41

Report generated by [PIT](#) 1.19.0

Fonte: Elaborado pelos autores (2026)

Os mutantes sobreviventes concentraram-se principalmente em duas categorias: (i) cálculos de valor monetário com tolerância decimal (multa, descontos), onde mutações em operadores matemáticos por vezes não alteram resultado dentro da margem de comparação usada nos `assertEquals` com `BigDecimal`; (ii) ramificações defensivas em código de validação, onde já houve verificação por DTO via Bean Validation - tornando alguns paths efetivamente inalcançáveis a partir do controller. Trabalho futuro inclui refinar os `assertEquals` para usar comparação exata e remover validações redundantes.

5.4 TEMPO DE EXECUÇÃO DAS SUÍTES

O tempo de execução das suites de teste é um indicador relevante para a sustentabilidade da estratégia: suites lentas tendem a ser executadas com menos frequência, reduzindo o valor do feedback automático. A Tabela 9 apresenta os tempos médios.

Tabela 9 - Tempo de Execução Médio das Suites

Suite	Tempo medio	Observacao
Unit tests (backend, JUnit)	~ 12 s	Sem contexto Spring
Integration tests (backend)	~ 35 s	Sobe contexto Spring + H2
Cucumber BDD (backend)	~ 18 s	Sobe contexto + cenários
PIT mutation testing	~ 8-12 min	Executado off-pipeline
Frontend Jest (full suite)	~ 15-20 s	Inclui coverage
Pipeline completo CI	~ 6-8 min	Inclui builds Vite/Electron

Fonte: Medido em hardware de desenvolvimento (2026)

Os tempos são considerados saudáveis. A suíte unitária pura termina em segundos, viabilizando feedback praticamente instantâneo durante o desenvolvimento. A suíte completa (incluindo integração e BDD) executa em aproximadamente um minuto, ainda dentro do limite psicológico aceitável para o desenvolvedor manter foco (BRADY; SHNEIDERMAN, 1980). O PIT - intencionalmente movido para execução offline - não afeta o ciclo diário de desenvolvimento.

5.5 DISCUSSÃO

Os resultados quantitativos demonstram que a estratégia adotada foi efetiva em produzir um sistema com indicadores de qualidade superiores aos tipicamente reportados em trabalhos acadêmicos comparáveis. Três observações merecem destaque.

Primeiro, a combinação de cobertura alta (89,5% global) com mutation score alto (76,6% nos services) é uma evidência robusta da qualidade dos testes. Como discutido na Seção 2.7, cobertura sozinha pode ser enganosa - mas cobertura alta acompanhada de mutation score alto e um indicador composto que combina dois ângulos diferentes: quanto código é exercitado pelos testes (cobertura) e quão sensíveis são os testes a alterações no código (mutação). Ambos altos implicam alta confiabilidade.

Segundo, a aplicação sistemática do padrão TFS tornou a construção da suíte de testes uma atividade rastreável e auditável. A presença da matriz de classes de equivalência no docstring de cada `*ServiceTest` serve como documentação viva do raciocínio de teste - documentação que persiste junto com o código e é atualizada quando novos casos de borda emergem. Trata-se de uma forma de transferência de conhecimento entre membros do time e entre gerações de desenvolvedores que mantém o projeto.

Terceiro, a integração do pipeline CI/CD com verificação automática de limites teve efeito disciplinador: durante o desenvolvimento, três tentativas de commit foram automaticamente revertidas pelo CI por não manterem o threshold de cobertura. Cada uma dessas reversões transformou-se em oportunidade pedagógica para o desenvolvedor entender qual ramificação havia sido esquecida no novo teste.

Como limitações da avaliação, registra-se: (a) ausência de testes E2E completos - somente integração via `MockMvc`; (b) cobertura BDD restrita ao

agregado Cliente; (c) ausência de testes de carga e desempenho - relegados a trabalho futuro; (d) execução do PIT em hardware único, sem análise de variância entre ambientes.

6 CONSIDERAÇÕES

O presente trabalho teve como objetivo o desenvolvimento de um Sistema de Aluguel de Trajes a Rigor, com ênfase na aplicação de práticas de engenharia de software, especificamente na garantia da qualidade através de testes automatizados. Face aos resultados obtidos ao longo do desenvolvimento, conclui-se que o objetivo central foi alcançado com a entrega de uma aplicação funcional estruturada de forma robusta.

Do ponto de vista da Engenharia de Software, o projeto demonstrou a viabilidade e a importância da integração de testes numa arquitetura em camadas. A implementação do backend (utilizando Java e Spring Boot) e do frontend (utilizando React) permitiu a digitalização das rotinas da locadora, abrangendo os cadastros de clientes, gestão de inventário e o fluxo de locação. A adoção de ferramentas de validação, como testes unitários e testes de mutação (com recurso ao PITest), garantiu a integridade das principais regras de negócio do domínio, nomeadamente os cálculos de valores de aluguer e a verificação de disponibilidade do estoque, assegurando que o código entregue é fiável e resiliente a falhas.

O sistema desenvolvido apresenta algumas limitações de âmbito estrutural e de negócio. A aplicação foi concebida para uso estritamente interno (desktop/balcão) pelos operadores da loja. Por conseguinte, não contempla um portal web para acesso direto pelo cliente final (B2C) e não possibilita a realização de reservas online autónomas. Adicionalmente, o software não possui integração com gateways de pagamento (como sistemas de cartões ou PIX) nem automatização de emissão fiscal, restringindo o controle financeiro ao registo interno no banco de dados.

Uma segunda delimitação refere-se aos quatro cenários operacionais identificados como críticos para o negócio - a devolução fora do prazo, a devolução de itens com defeito, a tentativa de locação de traje já reservado e o controle de caixa. As regras de negócio correspondentes foram implementadas e validadas na camada de backend, com cobertura pelos testes de serviço (notadamente `AluguelServiceTest` e `DevolucaoServiceTest`). Contudo, a exposição completa desses fluxos na interface gráfica (frontend) não foi concluída nesta versão. As telas e interações que permitiriam ao operador acionar diretamente o registo de avarias na devolução, a confirmação de devolução com cobrança de multa e a sinalização visual de itens já reservados, bem como um módulo dedicado de fluxo de caixa -

com lançamento de entradas e saídas e fechamento diário, permanecem como trabalho futuro. Assim, ainda que as validações e os cálculos estejam assegurados no servidor, a operação ponta a ponta desses casos pela interface demanda desenvolvimento complementar.

Sendo assim, o trabalho comprova que a aplicação de métricas de qualidade de código (como a cobertura de testes e o mutation score) não é apenas um rigor acadêmico, mas uma prática essencial para prevenir regressões financeiras e operacionais no sistema de gestão de uma pequena empresa.

REFERÊNCIAS

ASTELS, D. Test-Driven Development: A Practical Guide. Upper Saddle River: Prentice Hall, 2003.

BECK, K. Test-Driven Development by Example. Boston: Addison-Wesley, 2003.

BEIZER, B. Software Testing Techniques. 2. ed. New York: Van Nostrand Reinhold, 1990.

BOEHM, B. W. Software Engineering Economics. Englewood Cliffs: Prentice Hall, 1981.

BRADY, J. T.; SHNEIDERMAN, B. Response Time and Display Rate: A Survey of the Literature. Proceedings of the Association for Computing Machinery Annual Conference, 1980.

BURNSTEIN, I. Practical Software Testing: A Process-Oriented Approach. New York: Springer-Verlag, 2003.

COHN, M. Succeeding with Agile: Software Development Using Scrum. Boston: Addison-Wesley, 2009.

COLES, H. PIT: A Practical Mutation Testing Tool. Disponível em: <<https://pitest.org>>. Acesso em: mar. 2026.

CUCUMBER TEAM. Cucumber Reference. Disponível em: <<https://cucumber.io/docs/cucumber>>. Acesso em: abr. 2026.

DEMILLO, R. A.; LIPTON, R. J.; SAYWARD, F. G. Hints on Test Data Selection: Help for the Practicing Programmer. IEEE Computer, v. 11, n. 4, p. 34-41, 1978.

DODDS, K. C. Common Mistakes with React Testing Library. Disponível em: <<https://kentcdodds.com/blog/common-mistakes-with-react-testing-library>>. Acesso em: mar. 2026.

ELECTRON TEAM. Electron Documentation. Disponível em: <<https://www.electronjs.org/docs>>. Acesso em: abr. 2026.

FEATHERS, M. Working Effectively with Legacy Code. Upper Saddle River: Prentice Hall, 2004.

FOWLER, M. Continuous Integration. Disponível em: <<https://martinfowler.com/articles/continuousIntegration.html>>. Acesso em: mar. 2026. Publicado originalmente em 2006.

FOWLER, M. Mocks Aren't Stubs. Disponível em: <<https://martinfowler.com/articles/mocksArentStubs.html>>. Acesso em: mar. 2026. Publicado originalmente em 2007.

FOWLER, M. Test Coverage. Disponível em: <<https://martinfowler.com/bliki/TestCoverage.html>>. Acesso em: mar. 2026. Publicado originalmente em 2012.

FOWLER, M. GivenWhenThen. Disponível em: <<https://martinfowler.com/bliki/GivenWhenThen.html>>. Acesso em: mar. 2026. Publicado originalmente em 2013.

FOWLER, M. Refactoring: Improving the Design of Existing Code. 2. ed. Boston: Addison-Wesley, 2018.

GRAHAM, Dorothy; BLACK, Rex; VAN VEENENDAAL, Erik. Fundamentos do Teste de Software: Certificação ISTQB. 4. ed. São Paulo: Cengage Learning, 2019.

GIL, A. C. Como Elaborar Projetos de Pesquisa. 6. ed. São Paulo: Atlas, 2017.

HUMBLE, J.; FARLEY, D. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston: Addison-Wesley, 2010.

IEEE. IEEE Standard Glossary of Software Engineering Terminology - IEEE Std 610.12-1990. New York: IEEE, 1990.

ISO/IEC. ISO/IEC 25010:2011 - Systems and Software Engineering: Systems and Software Quality Requirements and Evaluation (SQuaRE). Geneva: ISO, 2011.

JACOCO TEAM. JaCoCo Documentation. Disponível em: <<https://www.jacoco.org/jacoco>>. Acesso em: abr. 2026.

JEST TEAM. Jest - Delightful JavaScript Testing. Disponível em: <<https://jestjs.io>>. Acesso em: abr. 2026.

JUNIT TEAM. JUnit 5 User Guide. Disponível em: <<https://junit.org/junit5>>. Acesso em: abr. 2026.

LAUDON, K. C.; LAUDON, J. P. Sistemas de Informacao Gerenciais. 15. ed. Sao Paulo: Pearson, 2017.

LWIN, M. M.; KHIN, P. P. Analysis of Code Coverage in Software Testing. International Journal of Computer Trends and Technology, v. 4, n. 9, 2013.

MARTIN, R. C. Design Principles and Design Patterns. Disponível em: <<http://www.objectmentor.com>>. Acesso em: mar. 2026. Publicado originalmente em 2000.

MARTIN, R. C. Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River: Prentice Hall, 2008.

MARTIN, R. C. The Clean Coder: A Code of Conduct for Professional Programmers. Upper Saddle River: Prentice Hall, 2011.

MARTIN, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Upper Saddle River: Prentice Hall, 2017.

MESZAROS, G. xUnit Test Patterns: Refactoring Test Code. Boston: Addison-Wesley, 2007.

MOCKITO TEAM. Mockito Framework Documentation. Disponível em: <<https://site.mockito.org>>. Acesso em: abr. 2026.

MYERS, G. J. The Art of Software Testing. New York: John Wiley & Sons, 1979.

MYERS, G. J.; SANDLER, C.; BADGETT, T. The Art of Software Testing. 3. ed. Hoboken: Wiley, 2011.

NORTH, D. Introducing BDD. Better Software Magazine, mar. 2006. Disponível em: <<https://dannorth.net/introducing-bdd>>. Acesso em: mar. 2026.

PETROVIC, G.; IVANKOVIC, M. State of Mutation Testing at Google. Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice. Gothenburg: ACM, 2018.

PRESSMAN, R. S.; MAXIM, B. R. Engenharia de Software: Uma Abordagem Profissional. 8. ed. Porto Alegre: AMGH, 2016.

REACT TEAM. React Documentation. Disponível em: <<https://react.dev>>. Acesso em: abr. 2026.

SOMMERVILLE, I. Engenharia de Software. 10. ed. São Paulo: Pearson, 2018.

SPRING TEAM. Spring Boot Reference Documentation - Version 3.4. Disponível em: <<https://docs.spring.io/spring-boot/docs/3.4/reference>>. Acesso em: abr. 2026.

TESTING LIBRARY TEAM. Testing Library Documentation. Disponível em: <<https://testing-library.com>>. Acesso em: abr. 2026.

TYPESCRIPT TEAM. TypeScript Documentation. Disponível em: <<https://www.typescriptlang.org/docs>>. Acesso em: abr. 2026.

VITE TEAM. Vite Guide. Disponível em: <<https://vitejs.dev/guide>>. Acesso em: abr. 2026.

WAKE, W. C. Arrange-Act-Assert: A Pattern for Writing Good Tests. Disponível em: <<https://xp123.com/articles/3a-arrange-act-assert>>. Acesso em: mar. 2026. Publicado originalmente em 2001.

WOHLIN, Claes et al. Experimentation in software engineering. Berlin: Springer, 2012.

WYNNE, M.; HELLESJOY, A. The Cucumber Book: Behaviour-Driven Development for Testers and Developers. 2. ed. Raleigh: Pragmatic Bookshelf, 2017.

YIN, R. K. Estudo de Caso: Planejamento e Métodos. 6. ed. Porto Alegre: Bookman, 2018.

APÊNDICE A - GLOSSÁRIO TÉCNICO

Este apêndice apresenta um glossário com os principais termos técnicos utilizados ao longo da monografia. As definições priorizam clareza didática e remetem, quando pertinente, a obras de referência listadas na seção REFERÊNCIAS.

Arrange-Act-Assert (AAA) - Padrão estrutural para escrita de testes unitários, organizando cada teste em três blocos sequenciais: preparação do estado e dos mocks, execução da ação testada e verificação do resultado.

Behavior-Driven Development (BDD) - Metodologia de desenvolvimento que enfatiza a especificação de comportamentos por meio de cenários em linguagem natural (tipicamente Gherkin), executáveis por ferramentas como Cucumber.

Clean Architecture - Padrão arquitetural proposto por Robert Martin que organiza o sistema em camadas concêntricas onde as dependências fluem apenas de fora para dentro, isolando regras de negócio de detalhes de infraestrutura.

Cobertura de código (code coverage) - Métrica que mede a fração do código de produção exercitada pelos testes automatizados. Pode ser medida em linhas, branches, condições ou caminhos.

Continuous Integration (CI) - Prática de engenharia que consiste em integrar o trabalho de cada desenvolvedor frequentemente, com verificação automática por build e suíte de testes a cada integração.

DTO (Data Transfer Object) - Objeto utilizado para transferir dados entre processos ou camadas, tipicamente sem comportamento, contendo apenas campos e assessores.

Gherkin - Linguagem usada pelo Cucumber para escrita de especificações executáveis em estrutura *Given-When-Then*. Suporta internacionalização - inclusive em português.

JaCoCo - Ferramenta de cobertura de código para JVM, baseada em instrumentação de bytecode. Gera relatórios em HTML, XML e CSV com cobertura de linhas, branches e instruções.

MockMvc - Componente do Spring Test que permite testar controllers MVC sem iniciar um servidor HTTP real, simulando requisições diretamente sobre o DispatcherServlet.

Mutation Testing - Técnica que mede a qualidade dos testes pela capacidade de detectar alterações sintáticas (mutantes) inseridas no código de produção. Quanto mais mutantes os testes 'matam', mais robustos eles são.

PCE - Particionamento em Classes de Equivalência - Técnica de derivação de casos de teste que divide o domínio de entrada em classes onde valores diferentes produzem o mesmo comportamento, permitindo testar com um representante de cada classe.

PIT (Pitest) - Ferramenta de mutation testing para JVM, otimizada por execução seletiva de testes que tocam cada mutante.

Specification (JPA) - Interface da Spring Data JPA que permite construir queries dinâmicas compostas, evitando proliferação de métodos de *repository* para cada combinação de filtros.

Test Doubles - Termo guarda-chuva, criado por Meszaros, para qualquer objeto sintético usado em lugar de uma dependência real em testes - inclui dummies, stubs, fakes, spies e mocks.

Test-Driven Development (TDD) - Prática de desenvolvimento em que o teste é escrito antes do código de produção, seguindo o ciclo Red-Green-Refactor em iterações curtas.

Testcontainers - Biblioteca Java que permite usar containers Docker como dependências em testes - tipicamente banco de dados, filas ou outros serviços.

TFS - Teste Funcional Sistemático - Padrão de teste adotado neste projeto que combina PCE e AVL em uma matriz rastreável de casos de teste numerados CTn, documentada no docstring do arquivo de teste.

V&V - Verificação e Validação - Conjunto de atividades que asseguram, respectivamente, que o software está sendo construído corretamente (verificação) e que está sendo construído o produto certo (validação).

APÊNDICE B - TRECHOS REPRESENTATIVOS DE CÓDIGO DE TESTE

Este apêndice reúne trechos de código mais extensos do que os incluídos no corpo do trabalho, oferecendo ao leitor uma visão concreta dos artefatos efetivamente produzidos no projeto. Os exemplos são representativos das quatro, três do back e uma do front, camadas de teste adotadas.

B.1 TESTE UNITÁRIO COMPLETO - TRAJESERVICETEST

```
/**
 * MATRIZ DE CLASSES DE EQUIVALENCIA - TrajeService
 * =====
 * Variavel      | Classes Validas (V)      | Invalidas (I)
 * -----|-----|-----
 * C1: descricao | V1 1..200 chars          | -
 * C2: nome      | V2 1..50 chars           | -
 * C3: valorItem | V3 > 0, precision 8/2    | -
 * C4: imageUrl  | V4a null / V4b preenchido | -
 * C5: id        | V5 existente             | I5 inexistente
 * C6: filtros   | V6a null/ V6b isolados/V6c | -
 * C7: busca     | V7a preenchida/V7b vazia/ | -
 *              | V7c null                 |
 *
 * VALORES LIMITE:
 * - descricao: 1 char (inf), 200 chars (sup)
 * - nome:      1 char (inf), 50 chars (sup)
 * - valorItem: 0.01 (inf), 999999.99 (sup)
 */
@ExtendWith(MockitoExtension.class)
@DisplayName("Testes de Comportamento do TrajeService (TFS)")
class TrajeServiceTest {

    @Mock
    private TrajeRepository repository;

    @InjectMocks
    private TrajeService service;

    @Test
    @DisplayName("CT1: deve criar quando todos campos validos")
    void ct1_deve_criar_quando_todosCamposValidosComImagemNula() {
        // Arrange
        TrajeRequest req = TrajeDataBuilder.umTraje()
            .semImageUrl()
            .buildRequest();
        when(repository.save(any(Traje.class)))
            .thenAnswer(inv -> {
                Traje t = inv.getArgument(0);
                t.setId(1L);
                return t;
            });

        // Act
        TrajeResponse resp = service.criar(req);

        // Assert
        assertThat(resp).isNotNull();
        assertThat(resp.id()).isEqualTo(1L);
    }
}
```

```

        verify(repository).save(any(Traje.class));
    }

    @Test
    @DisplayName("CT8: deve lancar ResourceNotFound quando id inexistente")
    void ct8_deve_lancarResourceNotFound_quando_idInexistente() {
        // Arrange
        when(repository.findById(999L)).thenReturn(Optional.empty());

        // Act + Assert
        assertThatThrownBy(() -> service.buscarPorId(999L))
            .assertInstanceOf(ResourceNotFoundException.class)
            .hasMessageContaining("Traje");
    }
}

```

B.2 TESTE DE INTEGRAÇÃO - CLIENTECONTROLLERINTEGRATIONTEST

```

@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
class ClienteControllerIntegrationTest {

    @Autowired private MockMvc mockMvc;
    @Autowired private ObjectMapper objectMapper;
    @Autowired private ClienteRepository repository;

    @BeforeEach
    void setUp() {
        repository.deleteAll();
    }

    @Test
    void deve_retornar201_quando_dadosValidos() throws Exception {
        ClienteRequest req = ClienteDataBuilder.umCliente()
            .buildRequest();

        mockMvc.perform(post("/clientes")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(req)))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.id").exists())
            .andExpect(jsonPath("$.nome").value("Joao da Silva"));

        assertThat(repository.findByCpfCnpj("12345678901")).isPresent();
    }

    @Test
    void deve_retornar409_quando_cpfDuplicado() throws Exception {
        // Arrange - cliente ja existente
        repository.save(
            ClienteDataBuilder.umCliente().buildEntity());

        ClienteRequest req = ClienteDataBuilder.umCliente()
            .buildRequest(); // mesmo CPF default

        mockMvc.perform(post("/clientes")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(req)))
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.mensagem")
                .value(containsString("ja cadastrado"))));
    }
}

```

B.3 FEATURE CUCUMBER - CRIAR_CLIENTE.FEATURE

language: pt

Funcionalidade: Gerenciamento de clientes

Esquema do Cenario: Nao deve criar cliente quando campo obrigatorio ausente

Dado que nao existe cliente com cpf "<cpfCnpj>"

Quando envio uma requisicao de cadastro com os dados:

nome	cpfCnpj	email	celular
<nome>	<cpfCnpj>	<email>	<celular>

Entao o status da resposta deve ser 400

E o campo "mensagem" da resposta deve conter "<mensagem>"

Exemplos:

nome	cpfCnpj	email	celular	mensagem
	12345678901	t@t.com	11999999999	Nome eh obrigatorio
Teste		t@t.com	11999999999	CPF eh obrigatorio

Cenario: Deve criar cliente quando todos os dados sao validos

Dado que nao existe cliente com cpf "12345678901"

Quando envio uma requisicao de cadastro valida

Entao o status da resposta deve ser 201

E deve existir um cliente com cpf "12345678901"

B.4 TESTE DE COMPONENTE REACT - BOTAO.TEST.TSX

```
import { render, screen, fireEvent } from '@testing-library/react';
import { Botao } from './Botao';
```

```
describe('Botao', () => {
  it('deve renderizar children corretamente', () => {
    render(<Botao>Clique aqui</Botao>);
    expect(screen.getByRole('button'))
      .toHaveTextContent('Clique aqui');
  });

  it('deve estar disabled quando disabled for true', () => {
    render(<Botao disabled>Botao</Botao>);
    expect(screen.getByRole('button')).toBeDisabled();
  });

  it('deve chamar onClick quando clicado', () => {
    const handleClick = jest.fn();
    render(<Botao onClick={handleClick}>Botao</Botao>);
    fireEvent.click(screen.getByRole('button'));
    expect(handleClick).toHaveBeenCalledTimes(1);
  });
});
```

APÊNDICE C - INVENTÁRIO COMPLETO DE ARQUIVOS DE TESTE

Este apêndice apresenta o inventário detalhado dos arquivos de teste do projeto, com nome completo do arquivo e número de métodos Test (no backend) ou it()/test() (no frontend).

C.1 BACKEND - ARQUIVOS *SERVICETEST

Tabela C.1 - Service Tests do Backend

Arquivo	Metodos
AluguelServiceTest.java	34
ClienteServiceTest.java	38
ContratoPdfServiceTest.java	43
DevolucaoServiceTest.java	14
ImagemServiceTest.java	34
MedidaServiceTest.java	24
TrajeServiceTest.java	29
TOTAL	216

Fonte: Contagem direta no código (2026)

C.2 BACKEND - ARQUIVOS *CONTROLLERTEST

Tabela C.2 - Controller Tests (unitarios) do Backend

Arquivo	Metodos
AluguelControllerTest.java	31
ClienteControllerTest.java	17
DevolucaoControllerTest.java	10
EnumControllerTest.java	1
ImagemControllerTest.java	7
MedidaControllerTest.java	23
TrajeControllerTest.java	22
TOTAL	111

Fonte: Contagem direta no código (2026)

C.3 FRONTEND - DISTRIBUIÇÃO POR CAMADA

Tabela C.3 - Inventário do Frontend por Camada

Camada / Diretorio	Arquivos
application/alugueis/	9
application/clientes/	7
application/trajes/	4
infrastructure/api/	3
domain/erros/	3
interfaces-graficas/componentes/	8
interfaces-graficas/paginas/	6
TOTAL	40

Fonte: Contagem direta no código (2026)

APÊNDICE D - INTERFACES-GRÁFICAS

Esta entrevista teve como objetivo homologar as interfaces gráficas de alto nível desenvolvidas para a aplicação desktop e verificar a adequação dos fluxos do sistema às rotinas diárias do estabelecimento. Durante o encontro, cada uma das telas apresentadas no Apêndice D foi exibida em funcionamento e submetida à avaliação direta da proprietária, responsável pela operação do negócio. As perguntas foram organizadas por tela, seguidas de uma avaliação geral da solução e de uma comparação com o processo de trabalho anterior, baseado no sistema anterior e papel. A conversa foi conduzida de forma aberta, e as respostas foram transcritas mantendo a linguagem coloquial da entrevistada.

D.1 INTERFACE GRÁFICA - Ficha do Cliente e Registro de Medidas

Cadastrar Novo Cliente

Dados Pessoais:

TIPO* CPF*

Pessoa Física

NOME*

SEXO ☐ Masculino ☒ Feminino

CEP* ENDEREÇO*

00000-000

NUM.* CIDADE* BAIRRO*

COMPL. U.F.*

E-MAIL*

CELULAR* FIXO

(00) 00000-0000 (00) 0000-0000

MEDIDAS em cm:

CINTURA 0,00	MANGA 0,00	ALTURA BUSTO 0,00
BASTO 0,00	CORPO 0,00	OMERO 0,00
DECOTE 0,00	QUADRIL 0,00	COMP. VESTIDO 0,00

Cancelar Salvar

© 2025 Sistema Interno

ENTREVISTADOR: Essa tela mostra a ficha do cliente: os dados de contato e as medidas dele. O que a senhora achou de ter tudo isso reunido num lugar só?

PROPRIETÁRIA: Achei muito bom. Antes a informação do cliente ficava tudo espalhada, um dado num caderno, a medida no sistema antigo... Ter tudo junto na ficha dele facilita demais o atendimento.

ENTREVISTADOR: A tela guarda o histórico de medidas do cliente. Isso faz diferença pra senhora?

PROPRIETÁRIA: Faz muita diferença, viu? Tem cliente que volta todo ano. Antes, toda vez que ele voltava, eu tinha que medir tudo de novo. Com a medida já guardada na ficha, eu só confiro e pronto, não preciso refazer aquele trabalho todo.

ENTREVISTADOR: E o jeito que o cadastro tá montado, com o contato e as medidas, ficou fácil de ler pra senhora?

PROPRIETÁRIA: Ficou, sim. Tá tudo organizadinho, eu acho a informação ráido. Essa tela eu aprovo do jeito que tá.

D.2 INTERFACE GRÁFICA - Fluxo de Registro de novo aluguel

TCC - Sistema de Locação de Trajes
Arquivo Editar Visualizar Janela Ajuda

Painel Administrativo - Sistema Interno

[← Voltar](#) **Realizar Aluguel**
Selecione um cliente e trajes para registrar um novo aluguel

Selecao de Cliente

CPF:

Nome:

Selecao de Trajes

Pesquisar Traje:

Trajes Selecionados (0)
Nenhum traje selecionado

Detalhes do Aluguel

Data de Retirada *

Data de Devolução *

Ocasão:

Valor Desconto (R\$):

Observações:

Subtotal	R\$ 0,00
Desconto	- R\$ 0,00
Total	R\$ 0,00

© 2005 Sistema Interno

ENTREVISTADOR: Essa aqui é a tela do principal serviço da loja, que é registrar um aluguel novo. Ela junta tudo numa tela só: escolher o cliente, marcar as datas e ir adicionando as peças. O que a senhora achou desse fluxo?

PROPRIETÁRIA: Ah, achei prático. Antes eu anotava tudo no papel e depois passava para o sistema anterior. Agora o sistema já soma sozinho, e acontece tudo numa tela só, sem eu ter que ficar pulando de um lugar pro outro.

ENTREVISTADOR: Tem aqui um resumo financeiro que o sistema calcula sozinho. O que a senhora achou disso?

PROPRIETÁRIA: Pois é, esse foi o que eu mais gostei! Antes ficava confuso visualizar no sistema antigo, agora calcula e exibe de uma vez. Dá mais segurança pra mim e pro cliente também.

ENTREVISTADOR: E pra marcar o período do aluguel, a data de retirada e a de devolução, ficou claro aqui na tela?

PROPRIETÁRIA: Ficou. Escolher o intervalo de datas ali direto ajuda muito a não embolar locação, ainda mais na época de formatura e casamento, que é quando mais aluga e quando eu mais me perdia com as datas.

ENTREVISTADOR: Então a senhora homologa esse fluxo de novo aluguel, acha que tá adequado pro atendimento no balcão?

PROPRIETÁRIA: Aprovo, com certeza. Pra mim essa é a tela mais importante de todas, e ela resolve bem o maior problema que eu tinha, que era a conta e aquela bagunça com as datas.

D.3 INTERFACE GRÁFICA - Cadastro Gestão de traje

The screenshot shows the 'Cadastrar Novo Traje' form in the 'TCC - Sistema de Locação de Trajes' application. The form is titled 'Cadastrar Novo Traje' and includes a 'Voltar' button. The form fields are as follows:

- Nome ***: Text input field.
- Descrição**: Text area.
- Tecido ***: Dropdown menu with 'Selecione'.
- Cor ***: Dropdown menu with 'Selecione'.
- Sexo ***: Dropdown menu with 'Selecione'.
- Tipo do traje ***: Dropdown menu with 'Selecione'.
- Preço (R\$) ***: Text input field with '0,00'.
- Tamanho ***: Dropdown menu with 'Selecione'.
- Textura ***: Dropdown menu with 'Selecione'.
- Status ***: Dropdown menu with 'Selecione'.
- Estampa**: Dropdown menu with 'Selecione'.
- Condição ***: Dropdown menu with 'Selecione'.
- Imagem do traje**: Image upload section with a placeholder 'Nenhuma imagem selecionada' and a 'Selecionar Imagem' button.

At the bottom right, there are 'Cancelar' and 'Salvar' buttons. The footer of the application shows '© 2025 Sistema Interno'.

ENTREVISTADOR: Essa é a tela que a senhora usa pra cadastrar uma peça nova no estoque. Tem os campos dos dados financeiros e das características da peça. Os campos dão conta de tudo que a senhora precisa anotar de um traje?

PROPRIETÁRIA: Dão sim. Tem o valor, tem o tamanho, a cor, a categoria... É isso que eu preciso pra saber quanto cobrar e onde aquela peça se encaixa.

ENTREVISTADOR: E faltou alguma coisa? Algum campo, alguma informação que pra senhora seria importante ter aqui?

PROPRIETÁRIA: Olha, assim de cara não senti falta de nada não. Pro jeito que eu trabalho tá completo. Pra mim essa tela tá boa.

ENTREVISTADOR: A senhora ficou tranquila de que, cadastrando a peça por aqui, ela vai aparecer certinha lá no catálogo e com o preço certo?

PROPRIETÁRIA: Fiquei. Como o preço e as características a gente já põe na hora do cadastro, depois não tem erro: a peça aparece lá com a informação certinha.

ANEXO E - SAÍDA DO COMANDO ./GRADLEW TEST

Este anexo reproduz, fielmente, a saída produzida pelo comando './gradlew test' executado sobre a versão auditada do projeto. A saída confirma o número de testes executados, o resultado e a cobertura JaCoCo.

```
$ ./gradlew test

> Task :compileJava
> Task :compileTestJava
> Task :test

=====

RESUMO DOS TESTES

=====

Total:   579
Passados: 579
Falhados: 0
Pulados: 0

=====

=====
COBERTURA DE CODIGO (JaCoCo)
=====
Linhas:  89,5% (1155/1290)
Branches: 68,2% (116/170)
=====

> Task :jacocoTestReport
> Task :jacocoTestCoverageVerification

BUILD SUCCESSFUL in 1m 24s
8 actionable tasks: 8 executed
```

ANEXO F - ROTEIRO E REGISTRO DA ENTREVISTA DE LEVANTAMENTO DE REQUISITOS

O levantamento de requisitos apoiou-se em entrevista semiestruturada com a proprietária do estabelecimento de referência, conduzida presencialmente na própria loja. Por se tratar de uma conversa de trabalho, e não de uma sessão gravada em áudio, as respostas reproduzidas a seguir não constituem transcrição literal: foram consolidadas pelos autores a partir das anotações tomadas durante o(s) encontro(s) e, em seguida, revisadas e validadas pela própria entrevistada, que confirmou refletirem fielmente o que foi relatado. O formato Pergunta/Resposta foi adotado para organizar o registro e torná-lo rastreável. As perguntas são abertas e exploram o processo de trabalho da loja; a tradução das necessidades relatadas em requisitos formais (RF e RNF) é trabalho dos autores e está consolidada na Tabela 1 da seção 4.1. As perguntas estão agrupadas em seis blocos temáticos, que acompanham os cinco agregados do domínio (Cliente, Traje, Aluguel, Medida e Devolução).

BLOCO I — O NEGÓCIO E O PROCESSO DE TRABALHO ATUAL

P1. Para começarmos, conte um pouco sobre a locadora: que tipo de cliente e de evento a senhora costuma atender?

R1. Atendo bastante casamento, o noivo, os padrinhos, às vezes o pai da noiva, e muita formatura no fim do ano. Também tem baile, festa de gala, jantar de empresa, esse tipo de evento mais formal. Vem cliente de todo tipo: a pessoa que aluga uma peça só e a empresa que precisa vestir uma equipe inteira para um evento. Cada época do ano tem um movimento diferente: a temporada de casamento é uma loucura, depois vem a de formatura, e isso muda bastante a forma como eu preciso me organizar.

P2. E como a senhora controla hoje os clientes, as peças e os aluguéis? A senhora usa algum sistema?

R2. Uso um sistema, sim, mas é um sistema antigo e genérico, desses de gestão de loja comum, não foi feito para locação de roupa. Ele até serve para guardar uns dados de cliente e algumas coisas de produto, mas para o jeito que a locação funciona, medida do corpo, peça reservada para uma data, contrato de

aluguel, devolução, ele não ajuda em nada. O resultado é que eu faço um monte de coisa por fora: ficha de medida no papel, uma agenda de mesa para anotar os aluguéis, o contrato no Word. Ou seja, eu tenho o sistema e ainda tenho o papel do lado, e os dois não conversam.

P3. E esse sistema que a senhora usa hoje, o que mais te incomoda nele?

R3. Várias coisas. Primeiro, ele é cheio de função que eu não uso para nada, tem menu, botão e tela de coisa que não tem nada a ver com o meu negócio, e eu me perco no meio daquilo tudo só para achar o pouco que eu preciso. Segundo, a tela é confusa, poluída, é informação espalhada, campo demais, e eu nunca sei direito onde clicar. Terceiro, e isso me cansa muito, as cores dele são ruins de enxergar: é tudo meio apagado, sem contraste, e eu forço a vista o dia inteiro. No fim, um sistema que era para me poupar tempo me faz perder tempo, tem hora que é mais fácil confiar no caderno do que naquela tela.

P4. E já aconteceu de tudo isso te causar um problema sério na frente do cliente, perder uma informação, cometer um erro? Pode contar um caso?

R4. Já aconteceu várias vezes. O caso que mais me marcou foi de um cliente antigo que voltou para alugar de novo e eu não achei a ficha de medida dele, como a medida fica tudo no papel, a ficha tinha se perdido na gaveta. Ele teve que voltar outro dia só para medir tudo de novo, e ficou contrariado. Também já reservei a mesma roupa para duas pessoas no mesmo fim de semana sem perceber, porque o sistema não controla isso e eu acabei anotando em duas páginas diferentes da agenda. E contrato, então: já perdi folha, já fiz conta errada na pressa. Cada erro desse é um cliente irritado e, às vezes, um cliente que não volta mais.

BLOCO II — CADASTRO DE CLIENTES E MEDIDAS

P5. Que informações a senhora precisa guardar de cada cliente?

R5. O básico é o nome, o telefone e o documento, CPF para pessoa física e CNPJ quando é empresa, porque tem empresa que aluga para a equipe. O e-mail também, para mandar confirmação. E o endereço completo, que eu uso no contrato. Eu preciso conseguir cadastrar um cliente novo, depois corrigir quando muda um telefone ou um endereço, ver a lista de quem são meus clientes e, se for o caso, remover alguém que pediu para sair da base. O sistema que eu uso tem uma tela de

cliente, mas é cheia de campo que não serve para nada no meu caso, e é confusa de preencher, então boa parte eu acabo anotando também por fora, por garantia.

P6. Como a senhora lida com as medidas do corpo do cliente? As medidas de homem e de mulher são diferentes?

R6. São bem diferentes. No homem eu meço cintura, manga, colarinho, tórax, a barra da calça, coisa de terno, de paletó. Na mulher é outra história: cintura e manga também, mas tem busto, ombro, quadril, decote, o comprimento do vestido... é um conjunto de medidas próprio. Como o sistema que eu uso não tem onde registrar medida nenhuma, hoje cada cliente tem uma ficha de papel, que eu guardo porque cliente bom volta e eu não quero medir tudo de novo toda vez. O sistema novo teria que entender que medida de homem e medida de mulher não são a mesma coisa.

P7. Quando um cliente que já alugou volta à loja, como a senhora encontra o cadastro dele?

R7. No sistema eu só consigo procurar pelo nome, e a busca é ruim, se eu não digito o nome exatamente como cadastrei, ele não acha. E o cliente às vezes usa um apelido, ou se apresenta de outro jeito. Eu queria muito poder procurar pelo CPF, que é o que não muda, ou pelo e-mail. Achar o cliente rápido é meio caminho andado: eu já vejo as medidas dele, o histórico, e não preciso começar tudo do zero.

P8. Já aconteceu de cadastrar o mesmo cliente duas vezes sem perceber?

R8. Aconteceu, e mais de uma vez. A pessoa volta meses depois, eu não lembro, cadastro de novo, e o sistema deixa, ele não reclama, não confere nada. Aí ficam dois cadastros da mesma pessoa, cada um com uma medida, um histórico pela metade. Vira uma confusão. O ideal era o sistema me avisar na hora: "esse CPF já existe", e não deixar eu duplicar.

BLOCO III — CATÁLOGO DE TRAJES

P10. Como é o seu acervo de peças hoje? Como a senhora descreve cada traje?

R10. Tenho bastante peça: terno, paletó, smoking, fraque, e do lado feminino vestido, saia, conjunto, macacão. Cada peça tem o tamanho, a cor, o tecido, a

estampa, o tipo, e o estado de conservação, porque roupa de aluguel desgasta, e eu preciso saber se aquela peça está nova, está boa, ou já está gasta demais para alugar. Cada uma também tem um valor. O sistema que eu uso tem um cadastro de produto genérico, que não foi pensado para roupa de locação, não dá para descrever a peça direito, com todos esses detalhes, então muita coisa acaba ficando na minha cabeça e em anotações soltas. Eu precisava de um catálogo de verdade: cadastrar a peça, atualizar quando muda alguma coisa, e dar baixa na que eu não alugo mais.

P11. Quando um cliente quer um modelo específico, um terno azul tamanho M, por exemplo, como a senhora encontra a peça certa?

R11. Na prática, no olho e na memória, eu vou até a arara e procuro. O sistema não me ajuda nisso: ele não tem um filtro decente e, mesmo que tivesse, ele não sabe o que é tamanho, cor ou tecido de traje. Se eu tenho muita peça, isso demora, e às vezes o cliente desiste de esperar. Eu queria poder filtrar: me mostra só os ternos azuis, tamanho M, que estão disponíveis. E, principalmente, que estejam livres na data do evento dele, não adianta achar a peça linda se ela já está alugada naquele fim de semana.

P12. A foto da peça ajudaria no sistema? Como a senhora usa as fotos hoje?

R12. Ajudaria muito. O sistema que eu uso não guarda foto nenhuma, então hoje eu tiro foto das peças no meu celular e fica tudo solto na galeria, sem organização nenhuma. Se cada peça no sistema tivesse a foto dela junto, eu mostrava na tela para o cliente, ele já via o modelo, a cor de verdade. Para mim e para ele seria bem mais fácil do que ficar caçando na arara.

P13. E esses atributos das peças, cor, tecido, tamanho, tipo, a senhora gostaria que fossem opções fixas ou texto livre? R13. Opções fixas, com certeza. No sistema de hoje é tudo campo de texto livre, e aí cada dia eu escrevo de um jeito, "azul", "azul-marinho", "azul escuro", e depois não consigo nem filtrar direito. Se as cores, os tecidos, os tamanhos, os tipos de peça e até os tipos de evento forem uma lista pronta para eu escolher, fica tudo padronizado, e quem mais usar o sistema vai cadastrar do mesmo jeito que eu.

BLOCO IV — O PROCESSO DE ALUGUEL

P14. Descreva, do começo ao fim, como a senhora faz um aluguel hoje.

R14. O cliente chega, a gente escolhe as peças, eu tiro ou confirmo as medidas, e combinamos as datas, o dia em que ele retira e o dia em que ele devolve. Como o sistema não foi feito para locação, o aluguel em si eu acabo anotando na agenda, e o contrato faço à parte. Parece simples, mas é onde mais erro acontece: data trocada, peça que eu achava livre e não estava, conta errada.

P15. Um mesmo cliente costuma levar mais de uma peça para o mesmo evento?

R15. Quase sempre. Num casamento, o noivo leva o terno, mas os padrinhos levam os deles, o pai da noiva também, às vezes saem cinco, seis peças no mesmo aluguel. Eu preciso registrar tudo isso junto, como um pacote só daquele cliente para aquele evento, e não uma peça de cada vez.

P16. Qual é o erro que mais te preocupa num aluguel

R16. Alugar a mesma peça para duas pessoas no mesmo fim de semana. Esse é o meu pesadelo. Já aconteceu: o cliente vem buscar o terno e ele não está, porque outro cliente levou. O sistema que eu uso nem enxerga esse tipo de coisa, ele não sabe que a peça já está reservada para uma data. Não tem como consertar na hora, é vergonhoso, e eu perco o cliente. Eu queria um sistema que simplesmente não me deixasse fazer isso, se a peça já está comprometida naquele período, ele tem que me avisar na hora e travar.

P17. E como a senhora sabe, hoje, se uma peça está livre numa certa data?

R17. Eu folheio a agenda de papel e procuro a peça naquelas datas, porque o sistema não controla isso. É lento e é falho, porque depende de eu ter anotado certo e de eu lembrar de olhar. O que eu queria era perguntar ao sistema "essa peça está livre do dia tal ao dia tal?" e ele me responder na hora, olhando todos os aluguéis. E que ele soubesse sozinho quando uma peça está alugada, sem eu ter que ficar marcando à mão.

P18. E quando o cliente quer mudar a data do evento, ou desistir do aluguel?

R18. Mudar a data acontece direto, o casamento muda, a formatura atrasa. Hoje eu rabisco a agenda e rezo para não me confundir. E desistência também acontece: o cliente cancela. Eu preciso poder alterar um aluguel que ainda não aconteceu, mudar a data, trocar uma peça, e poder cancelar quando for o caso; e, ao cancelar, as peças têm que voltar a ficar livres para outro cliente.

P19. Quando são várias peças, como a senhora calcula o valor total e os descontos?

R19. Na calculadora, somando o valor de cada peça na mão, o sistema não faz essa conta de locação. E, na pressa do atendimento, eu erro conta, já cobreí a menos e percebi depois, já cobreí a mais e o cliente reclamou. Quando eu dou um desconto, para cliente fiel ou para pacote grande, é mais uma conta na cabeça. O sistema teria que somar tudo sozinho e aplicar o desconto, para eu não errar.

P20. A senhora consegue ver com facilidade o que está programado, o que vai sair nesta semana, ou os aluguéis de um cliente?

R20. Não, e isso me faz falta todo dia. Como os aluguéis ficam na agenda de papel, para saber o que sai nesta semana eu tenho que ir folheando dia por dia. Eu queria abrir uma lista e ver: o que está para retirar, o que está para devolver, os aluguéis de um cliente específico, quantos casamentos eu tenho neste mês. Poder filtrar por data, por cliente, pelo tipo de evento, ou por aluguel que está em aberto ou já encerrado, isso me ajudaria a me organizar e a não ser pega de surpresa.

BLOCO V — DEVOLUÇÃO, CONTRATO E COBRANÇAS

P21. Como é a devolução da peça? O que a senhora confere quando o traje volta?

R21. Quando o cliente devolve, eu confiro peça por peça. Vejo se está tudo certo, se não tem mancha, rasgo, se não falta nada. Como o aluguel sai com várias peças, cada uma volta num estado, uma pode estar perfeita e outra precisar de cuidado. Eu preciso registrar a devolução com a situação de cada peça, uma por uma, e não o aluguel inteiro de uma vez só.

P22. E o que a senhora faz quando uma peça volta danificada, manchada ou precisando de reparo?

R22. Aí aquela peça não pode sair de novo na hora. Ela vai para a lavanderia, para o conserto, ou para a higienização, e fica fora de circulação até estar pronta. O sistema precisa entender isso: quando eu marco que a peça voltou avariada, ela não pode aparecer como disponível para outro cliente alugar. O estado da peça muda, e isso tem que travar o aluguel dela até eu liberar de novo.

P23. E quando o cliente atrasa a devolução?

R23. Atraso é comum, e é onde eu perco dinheiro. O cliente devolve depois do combinado e, na correria, eu simplesmente esqueço de cobrar a multa, ou cobro "no sentimento", sem critério nenhum, porque o sistema não calcula nada disso. E, enquanto a peça não volta, eu também não posso alugar para mais ninguém. Eu queria que o sistema calculasse a multa pelo atraso automaticamente, com uma regra clara, para eu cobrar certo e não deixar passar.

P24. Como é feito o contrato de aluguel hoje? O que ele precisa conter?

R24. O sistema que eu uso não gera contrato de locação nenhum, então hoje é na mão, num modelo de Word que eu vou preenchendo. Dá trabalho e fica com cara amadora. O contrato precisa ter os dados do cliente, nome, documento, endereço, contato, a lista das peças que ele está levando, as datas de retirada e devolução, o valor, e o espaço para a assinatura das duas partes. Eu queria que, depois de registrar o aluguel, o sistema já gerasse esse contrato pronto, com tudo certinho, para eu só imprimir e assinar.

P25. Depois que a peça volta e está tudo certo, como a senhora marca que ela está disponível de novo?

R25. Hoje? Na cabeça, porque o sistema não controla a situação da peça. Eu sei que voltou e que pode sair de novo. Mas isso é perigoso, porque, se eu esquecer, ou se outra pessoa estiver atendendo, a peça fica "sumida" do controle. Eu queria que, quando eu registrasse a devolução, a peça voltasse para "disponível" sozinha, e que, quando ela sai num aluguel, ela saísse de "disponível" sozinha também. Sem eu ter que lembrar de marcar cada coisa.

BLOCO VI — AMBIENTE DE USO E EXPECTATIVAS COM O SISTEMA

P26. Pensando no dia a dia: onde e como esse sistema seria usado na loja?

R26. No computador do balcão, durante o atendimento, com o cliente ali na minha frente. Não é uma coisa que eu vou usar em casa, com calma, é na hora, no movimento. Então tem que ser rápido e simples, porque eu não posso deixar o cliente esperando enquanto a tela carrega.

P27. A internet na loja é estável? O que acontece quando ela cai?

R27. Cai, sim, com uma certa frequência, é da região. E, quando cai, eu não posso parar de atender, o cliente está na minha frente. Por isso eu não queria um sistema que dependesse o tempo todo da internet para qualquer coisinha. Pelo menos consultar o catálogo, ver as peças, eu precisava conseguir fazer mesmo com a conexão ruim. Um programa instalado no computador me passa mais segurança do que uma página que só abre online.

P28. Para a senhora, o que seria um "sistema bom"? E o que ele não pode deixar acontecer de jeito nenhum?

R28. Sistema bom, para mim, é o que eu confio. Que não perde a informação que eu coloquei, que está sempre certo quando eu consulto. E que seja simples e limpo, só com o que eu preciso, a tela clara e fácil de enxergar, ao contrário do que eu tenho hoje. O que ele não pode deixar acontecer é justamente o que mais me dói: alugar a mesma peça duas vezes, perder a medida de um cliente, ou registrar uma devolução pela metade, devolver a peça, mas o aluguel continuar "em aberto", ou o contrário. Se eu registro alguma coisa, tem que ser registrado por inteiro, ou não registra. Não pode ficar pela metade, porque informação pela metade é o que me deixa na mão.

P29. Tem alguma coisa que a senhora gostaria, e que eu não perguntei?

R29. Acho que o resumo de tudo é esse: eu quero parar de depender de papel, de anotação por fora e de um sistema que mais atrapalha do que ajuda. Quero abrir um sistema só, feito para a locação, e ter ali, organizado, quem são meus clientes, o que eu tenho de peça, o que está alugado e para quando, o que vai voltar. Hoje eu trabalho com medo de errar. Se o sistema me der essa segurança, para mim já está ótimo.